Mammalia-Macaque-Dominance Network

Laura Silvana Alvarez Luque - Daniel Losada Molina

The aim of this project is to do a complete analysis of a network of interest. This includes: the network description, modeling through different techniques and results that can be obtained.

The selected network contains the dominance relations between *Macaca fuscata* females that were determined based on approach-retreat episodes around the food. The dominance range order was arranged based on these dyadic relations. Dominance is defined as "Physical contact unique to aggressive dominance interactions such as biting, head butting, fighting"

- **Vertices:** 62 Macaques.
- **Edges:** 1.167 edges representing the pairwise dominance between macaques.
- **Weights:** Frequency of the dominance.
- **Notes:** This is a animal interation network, undirected and weighted.

In [1]:
```
import zipfile
import base64
import random
import os
import pickle

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import igraph as ig
import plotly.graph_objects as go
import networkx as nx
import matplotlib.cm as cm
import matplotlib.colors as colors
import statsmodels.api as sm
from scipy.stats import expon, lognorm, poisson
from sklearn.manifold import MDS

from PIL import Image
```

```
from IPython.display import display
from collections import Counter

from igraph import *
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
from collections import defaultdict
```

Functions

In [2]:

```
def unzip_file(zip_path, extract_to):
    """
    Unzip a file from zip_path into the extract_to directory.
    """
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_to)

def get_node_info(g, node_id, threshold_pt=None):
    # Convert original ID to internal index
    internal_idx = g.vs.find(id=node_id).index

    if threshold_pt is not None:
        threshold = np.percentile(g.es["weight"], threshold_pt)
        print(f"Threshold for edge weight: {threshold}")

    incident_edges = g.incident(internal_idx, mode="ALL")
    count = 0
    node_strength = 0

    for eid in incident_edges:
        if threshold_pt is not None and g.es[eid]["weight"] < threshold:
            continue

        edge = g.es[eid]
        source = edge.source
        target = edge.target
        weight = edge["weight"]
        node_strength += weight

        source_id = g.vs[source]["id"]
        target_id = g.vs[target]["id"]
        print(f"Edge from {source_id} to {target_id}, weight = {weight}")
        count += 1

    print("Total incident edges (above threshold):", count)
    print(f"Total strength of node {node_id}:", node_strength)

def get_layout(g, layout_type="fr", seed=None):
    if seed is not None:
            np.random.seed(seed)
            np.random.seed(seed)
    if layout_type == "fr":
        print("Using Fruchterman-Reingold layout")
        return g.layout_fruchterman_reingold()
    elif layout_type == "kk":
        print("Using Kamada-Kaway layout")
        return g.layout_kamada_kawai()
    elif layout_type == "davidson_harel":
        print("Using Davidson-Harel layout")
        return g.layout_davidson_harel()
    elif layout_type == "graphopt":
```

```
        print("Using Graphopt layout")
        return g.layout_graphopt()
    else:
        return g.layout(layout_type)

def plotly_graph(g):
    # Convert to NetworkX
    G_nx = g.to_networkx()

    # Get layout positions
    pos = nx.spring_layout(G_nx)

    # Prepare edge lines
    edge_x = []
    edge_y = []
    for u, v in G_nx.edges():
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_x += [x0, x1, None]
        edge_y += [y0, y1, None]

    edge_trace = go.Scatter(
        x=edge_x,
        y=edge_y,
        line=dict(width=0.5, color='#888'),
        hoverinfo='none',
        mode='lines',
        showlegend=False
    )

    # Create "hover points" at edge midpoints
    edge_hover_x = []
    edge_hover_y = []
    edge_text = []

    for u, v, data in G_nx.edges(data=True):
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_hover_x.append((x0 + x1) / 2)
        edge_hover_y.append((y0 + y1) / 2)
        weight = data.get("weight", 1)
        edge_text.append(f"{u} — {v}<br>weight: {weight}")

    edge_hover_trace = go.Scatter(
        x=edge_hover_x,
        y=edge_hover_y,
        mode='markers',
        hoverinfo='text',
        text=edge_text,
        marker=dict(size=5, color='rgba(0,0,0,0)'),  # invisible
        showlegend=False
    )

    # Prepare node positions
    node_x = []
    node_y = []
    node_text = []
    for node in G_nx.nodes():
        x, y = pos[node]
        node_x.append(x)
        node_y.append(y)
        node_text.append(f"Node {node}")

    node_trace = go.Scatter(
```

```
        x=node_x,
        y=node_y,
        mode='markers+text',
        text=[str(n) for n in G_nx.nodes()],
        textposition='top center',
        hoverinfo='text',
        marker=dict(
            size=10,
            color='lightblue',
            line_width=1
        )
    )

    # Combine everything
    fig = go.Figure(
        data=[edge_trace, edge_hover_trace, node_trace],
        layout=go.Layout(
            title='Dominance Network of Macaques',
            showlegend=False,
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()

def get_weights_of_path(g, path):
    """
    Given a path, return the weights of the edges along that path.
    """
    original_weights = []
    inv_weights = []

    for i in range(len(path) - 1):
        source = path[i]
        target = path[i + 1]

        # Get edge ID between consecutive vertices
        edge_id = g.get_eid(source, target)

        # Extract weights
        w = g.es[edge_id]["weight"]
        iw = g.es[edge_id]["inv_weight"]

        original_weights.append(w)
        inv_weights.append(iw)

    # Print results
    print("Original weights on path:", original_weights)
    print("Inverted weights on path:", inv_weights)


def plot_degree_distribution(g, title="Degree Distribution", weighted=False):
    """
    Computes degrees (weighted or unweighted), plots their distribution,
    and returns degree info including max degree and corresponding vertex/vertices.
    """
    # Choose between weighted and unweighted degree
    if weighted:
        degrees = g.strength(weights=g.es["weight"])
    else:
        degrees = g.degree()
```

```
    # Compute degree distribution
    unique_degrees, counts = np.unique(degrees, return_counts=True)
    probabilities = counts / counts.sum()

    # Plot the distribution
    plt.figure(figsize=(8, 5))
    plt.scatter(unique_degrees, probabilities, color='black')  # Dots
    plt.vlines(unique_degrees, 0, probabilities, color='blue', alpha=0.6)  # Lines

    # Labels and title
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.title(title)

    # Show plot
    plt.show()

    # Get max degree and vertex/vertices
    max_degree = max(degrees)
    #vertices_with_max = [v.index for v, d in zip(g.vs, degrees) if d == max_degree
    vertices_with_max = [v['id'] for v, d in zip(g.vs, degrees) if d == max_degree]

    # Return degrees and distribution if needed later
    return degrees, unique_degrees, probabilities, max_degree, vertices_with_max

# Network Visualization Funcitons

def plot_simple_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return

    # Layout for node positions
    layout = get_layout(g,layout_type, 253)
    vertex_label=[str(i) for i in g.vs["id"]]

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g,
        layout=layout,
        vertex_label=vertex_label,  # show node indices
        edge_width=[w for w in g.es["weight"]],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)


def plot_weighted_graph(g, layout_type="fr", target=None, display_img=True):

    if target is not None and os.path.exists(target):
        print(f"[✓] Skipping {target} (already exists)")
        if display_img:
            img = Image.open(target)
            display(img)
        return
```

```
    g_copy = g.copy()
    # Layout for node positions
    layout = get_layout(g_copy,layout_type, 253)

    weights = np.array(g_copy.es["weight"])
    threshold = np.percentile(weights, 70)
    norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())

    # Compute node strength from strong edges
    node_strength = np.zeros(g_copy.vcount())
    for e in g_copy.es:
        if e["weight"] >= threshold:
            node_strength[e.source] += e["weight"]
            node_strength[e.target] += e["weight"]

    # Normalize node strength
    min_size, max_size = 20, 80  # Adjust for igraph scale
    normalized_strength = (node_strength - node_strength.min()) / (node_strength.ma
    node_sizes = min_size + (max_size - min_size) * normalized_strength
    node_colors = [colors.to_hex(cmap(val)) for val in normalized_strength]  # Conv

    # Set vertex attributes
    g_copy.vs["size"] = node_sizes
    g_copy.vs["color"] = node_colors
    g_copy.vs["label"] = g_copy.vs["name"]
    g_copy.vs["label_size"] = 10

    # Set edge attributes
    for e in g_copy.es:
        w = e["weight"]
        if w >= threshold:
            e["color"] = colors.to_hex(cmap(norm(w)))
            e["width"] = 1.5
            e["alpha"] = 0.6
        else:
            e["color"] = "#D3D3D3"  # Light gray for weak edges
            e["width"] = 0.3
            e["alpha"] = 0.2

    # Plot using igraph's built-in plot function
    plot_obj = ig.plot(
        g_copy,
        layout=layout,
        edge_color=g_copy.es["color"],
        edge_width=g_copy.es["width"],
        vertex_color=g_copy.vs["color"],
        vertex_size=g_copy.vs["size"],
        vertex_label=g_copy.vs["label"],
        vertex_label_size=g_copy.vs["label_size"],
        bbox=(800, 800),
        margin=50,
        target=target
    )

    if display_img:
        display(plot_obj)
```

In [3]:

```
%%capture
def plotly_graph_with_same_image(g, image_path='./imgs/macaco_agua_sin_fondo.png'):
    # Load and encode image as base64
    with open(image_path, "rb") as image_file:
        encoded_image = base64.b64encode(image_file.read()).decode()
        image_uri = f'{encoded_image}'
```

```
    # Convert to NetworkX
    G_nx = g.to_networkx()
    pos = nx.spring_layout(G_nx)

    # Prepare edge lines
    edge_x = []
    edge_y = []
    for u, v in G_nx.edges():
        x0, y0 = pos[u]
        x1, y1 = pos[v]
        edge_x += [x0, x1, None]
        edge_y += [y0, y1, None]

    edge_trace = go.Scatter(
        x=edge_x, y=edge_y,
        line=dict(width=0.5, color='#888'),
        hoverinfo='none', mode='lines',
        showlegend=False
    )

    # Prepare node positions and hover info
    node_x = []
    node_y = []
    node_text = []
    for node in G_nx.nodes():
        x, y = pos[node]
        node_x.append(x)
        node_y.append(y)
        node_text.append(f"Node {node}")

    node_trace = go.Scatter(
        x=node_x, y=node_y,
        mode='markers',
        hoverinfo='text',
        text=node_text,
        marker=dict(size=20, color='rgba(0,0,0,0)'),  # invisible
        showlegend=False
    )

    label_trace = go.Scatter(
        x=node_x,
        y=[y - 0.03 for y in node_y],
        mode='text',
        text=[str(n) for n in G_nx.nodes()],
        textposition='bottom center',
        hoverinfo='none',
        showlegend=False
    )

    # Add the same image to all nodes
    image_traces = []
    for x, y in pos.values():
        image_traces.append(
            dict(
                source=image_uri,
                xref="x", yref="y",
                x=x - 0.05, y=y + 0.05,
                sizex=0.1, sizey=0.1,
                xanchor="left", yanchor="top",
                sizing="stretch",
                opacity=1,
                layer="above"
            )
        )
```

```
    # Build the figure
    fig = go.Figure(
        data=[edge_trace, node_trace, label_trace],
        layout=go.Layout(
            title='Graph with Macaque Image on Each Node',
            images=image_traces,
            showlegend=False,
            hovermode='closest',
            margin=dict(b=20, l=5, r=5, t=40),
            xaxis=dict(showgrid=False, zeroline=False),
            yaxis=dict(showgrid=False, zeroline=False)
        )
    )

    fig.show()
```

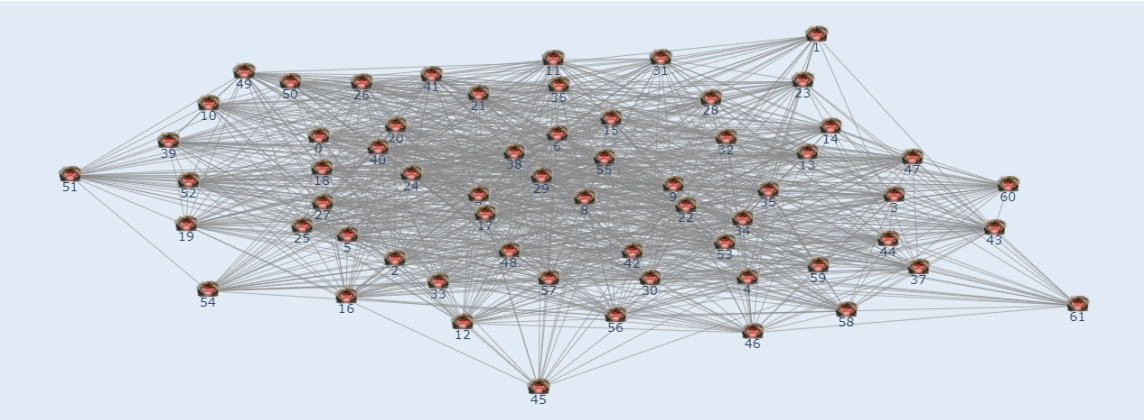
1. Input data and preliminary analysis

In [4]:
```
unzip_file('./mammalia-macaque-dominance.zip', './data')
```

In [5]:
```
#%%capture
g = ig.Graph.Read_Ncol("data/mammalia-macaque-dominance.edges", weights=True, direc
g.vs["id"] = [int(name) for name in g.vs["name"]]
#g.vs.find(id=4).index
print(g.summary())
```

```
IGRAPH UNW- 62 1167 --
+ attr: id (v), name (v), weight (e)
```

In [6]:
```
%%capture
#plotly_graph(g)
```



In [7]:
```
order = g.vcount()
size = g.ecount()

print(f"The order of the network is: {order} (vertices)")
print(f"The size of the network is: {size} (edges)")
```

```
The order of the network is: 62 (vertices)
The size of the network is: 1167 (edges)
```

We are dealing with a highly connected graph, as we can see by the edge-vertex ratio:

$$\frac{|E|}{|V|} = \frac{1167}{62} \approx 18.82$$

This means that on average, we have 18.82 edges per vertex. Therefore, it is not surprising to find that we only have one large component that contains all vertices. For this reason, we do not have any subnetworks of interest, and all analyses will be conducted on the entire network.

In [8]:
```
components = g.components()
print(f"Number of subnetworks: {len(components)}")
```

```
Number of subnetworks: 1
```

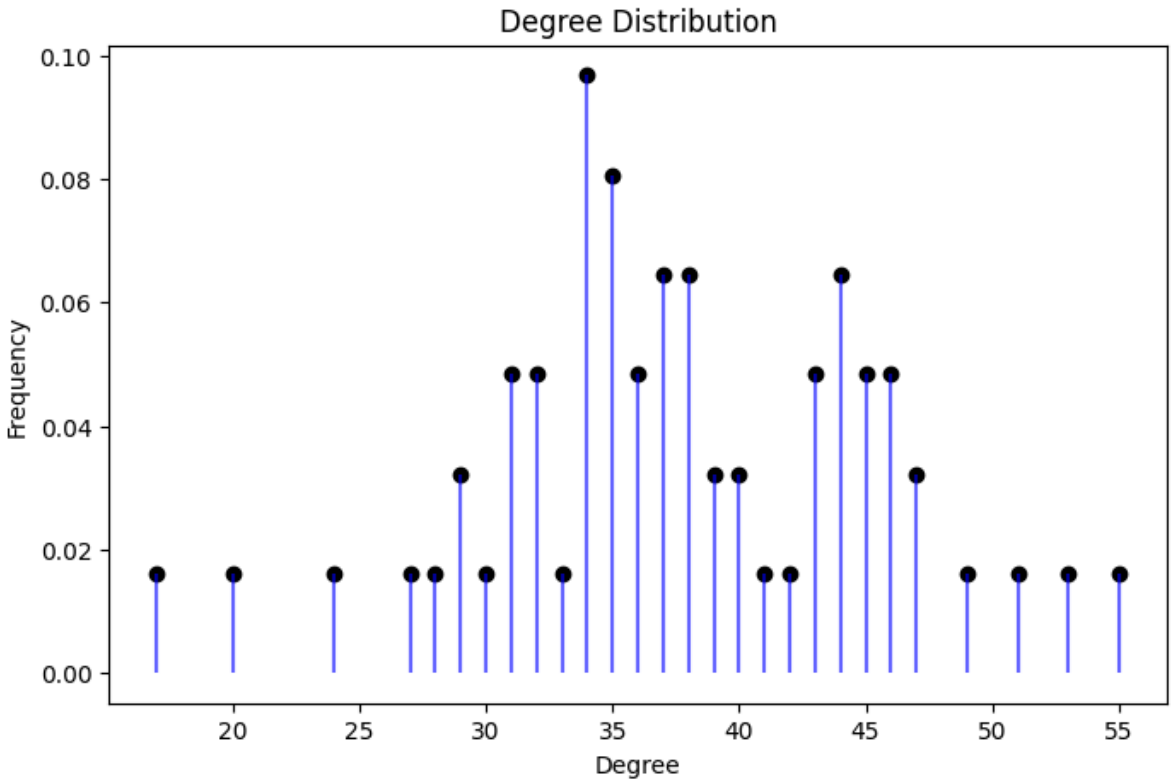
Due to the nature of our data, loops make no sense, as the edges represent interaction between two individuals. Anyway, we check that our data has zero edges with same source and target.

In [9]:
```
sum(g.is_loop())
```

Out[9]:
```
0
```

With the degree distribution, we can check that the network is highly connected, as we saw previously. It does not have the classical decreasing distribution starting in small degree values, instead, it starts in 14 and seems to have a bimodal distribution with concentrations around 34 and 44. Also, the most connected vertex is the 29 with 55 edges connected to it.

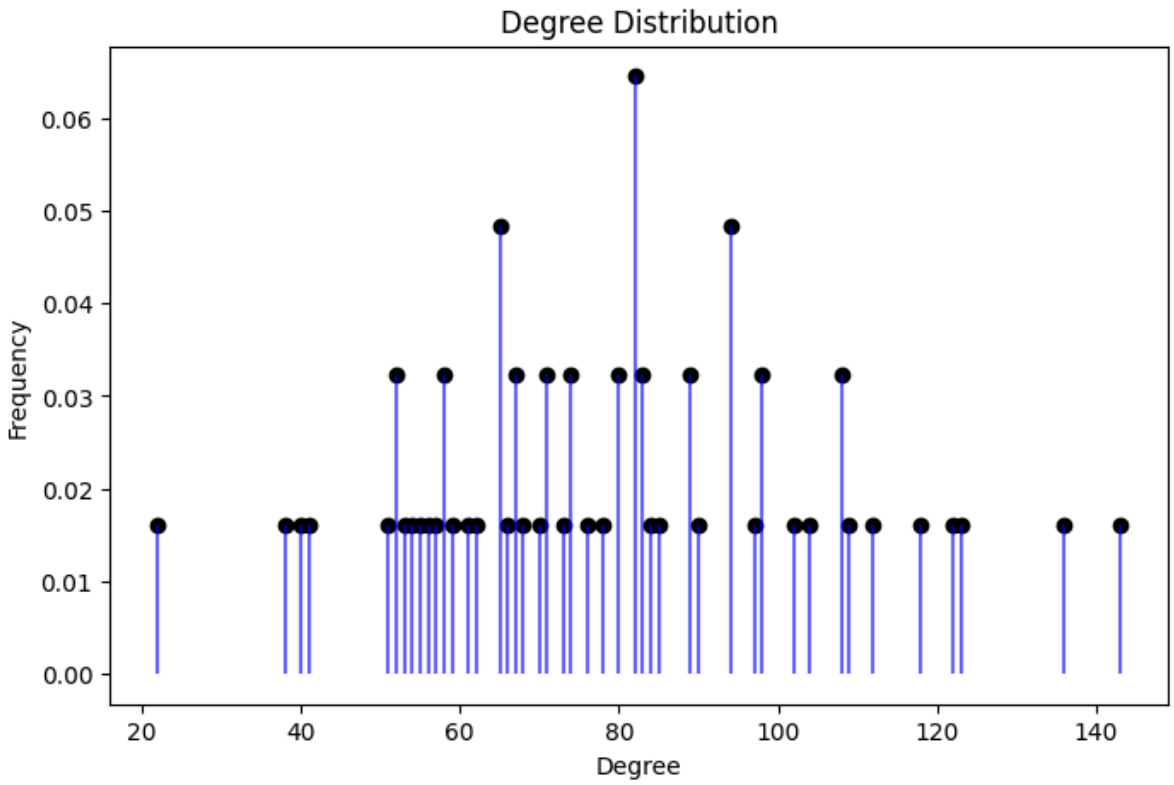
In [10]:
```
degrees, unique_deg, probs, max_deg, max_vertices = plot_degree_distribution(g)

print(f"Max degree: {max_deg}")
print(f"Vertices with max degree: {max_vertices}")
```




```
Max degree: 55
Vertices with max degree: [48]
```

Analyzing the weighted degree distribution, the bimodal distribution is lost, and the maximum is centered around 80. On the other hand, the minimum value remains large, and the most connected vertex is still the 29.

In [11]:
```
degrees_w, unique_deg_w, probs_w, max_deg_w, max_vertices_w = plot_degree_distribut

print(f"Max degree: {max_deg_w}")
print(f"Vertices with max degree: {max_vertices_w}")
```




```
Max degree: 143.0
Vertices with max degree: [48]
```

Next we are going to compute the diameter.

In [12]:
```
# Diameter (unweighted)
diameter = g.diameter(directed=False, weights=None)
print("Unweighted diameter:", diameter)

# Get the actual pair of farthest nodes and the path
farthest_pair_idx = g.get_diameter(directed=False, weights=None)
farthest_path_names = [g.vs[i]["id"] for i in farthest_pair_idx]

print("Farthest vertices (unweighted) igraph idx:", farthest_pair_idx)
print("Farthest vertices (unweighted original IDs):", farthest_path_names)
```

```
Unweighted diameter: 2
Farthest vertices (unweighted) igraph idx: [0, 1, 39]
Farthest vertices (unweighted original IDs): [1, 2, 29]
```

As we have already seen, our graph is highly connected, so that the unweighted diameter is 2 is quite intuitive. That means that between two individuals, there is a path of at most 2 (another individual in the middle). But it's not a special case to have the shortest distance of 2, in fact we have 1448 paths of length 2. This means that there are 1448 pairs of individuals that are not directly connected, but they are connected through another individual. This tells us a lot of how dominance interactions are distributed in the network. Even if two macaques never had a dominance interaction, they will have a third individual in common.

In [13]:
```
sp_lengths = g.distances(weights=None)  # or .shortest_paths()
```

```
distances = [dist for row in sp_lengths for dist in row if dist != 0]
dist_count = Counter(distances)
print(dist_count)
```

```
Counter({1: 2334, 2: 1448})
```

To compute the weighted diameter, we need to invert the weights, as in our case, higher weight means higher connection.

In [14]:
```
# Add inverse weights (handle division by zero if needed)
g.es["inv_weight"] = [1/w if w != 0 else float("inf") for w in g.es["weight"]]

# Then compute diameter using this new attribute
diameter_w = g.diameter(directed=False, weights="inv_weight")
path_w_idx = g.get_diameter(directed=False, weights="inv_weight")
path_w_names = [g.vs[i]["id"] for i in path_w_idx]

print("Weighted diameter (based on interaction):", diameter_w)
print("Farthest vertices (based on interaction) igraph idx:", path_w_idx)
print("Farthest path (based on interaction original IDs):", path_w_names)
```

```
Weighted diameter (based on interaction): 1.083333333333333
Farthest vertices (based on interaction) igraph idx: [45, 43, 56, 54]
Farthest path (based on interaction original IDs): [6, 61, 60, 46]
```

The longest shortest path is connected through a path whose total inverse-weight is 1.0833. Notice that using the weights now we obtain a farthest path that contains 3 edges that connect 4 individuals.

In [15]:
```
get_weights_of_path(g, path_w_idx)
```

```
Original weights on path: [3.0, 2.0, 4.0]
Inverted weights on path: [0.3333333333333333, 0.5, 0.25]
```

Finally, we compute the adjancency matrix and compare the results previously obtained. The results are the same, as expected. The adjacency matrix is symetric, as we have an undirected graph. The diagonal is zero, as we do not have loops. The sum of the rows is equal to the number of edges connected to that vertex, and the sum of the columns is equal to the number of edges that are connected to that vertex.

In [16]:
```
# Get the adjacency matrix
adj_matrix = np.array(g.get_adjacency().data)
adj_matrix_w = np.array(g.get_adjacency(attribute="weight").data)

# Print the adjacency matrix
print(adj_matrix)
print(adj_matrix_w)
```

```
[[0 1 1 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 [1 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 1 0]
 [0 0 0 ... 1 0 0]
 [0 0 0 ... 0 0 0]]
[[0. 3. 1. ... 0. 0. 0.]
 [3. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 4. 0.]
 [0. 0. 0. ... 4. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]
```

In [17]:

```python
# Results from the adjacence matrix

## degrees
degrees_adj = adj_matrix.sum(axis=0)

is_symmetric = np.allclose(adj_matrix, adj_matrix.T)

## Loops
self_loops = np.trace(adj_matrix)

## Nodes with degree 0 (isolated)
isolated = np.where(adj_matrix.sum(axis=1) == 0)[0]

print(degrees_adj)
print(is_symmetric)
print(self_loops)
print(isolated)
```

```
[38 24 44 36 38 41 49 46 46 47 31 38 34 35 36 42 35 53 40 35 45 39 35 32
 47 40 35 44 33 55 43 32 39 31 45 43 37 32 51 34 46 37 45 37 34 17 29 38
 44 34 31 30 34 43 27 36 37 44 29 34 28 20]
True
0
[]
```

The density is defined as:

$$Density = \frac{2 \times edges}{n(n-1)}$$

So is making a relation between the connections and the number of vertices. A high number means a dense network, while a small number a sparse one.

In [18]:

```python
# Compute and print density
density = g.density(loops=False)
print(f"\nNetwork density: {density:.4f}")

# Interpret
if density > 0.5:
    print("This is a dense network.")
elif density < 0.1:
    print("This is a sparse network.")
else:
    print("This is a moderately connected network.")
```

```
Network density: 0.6171
This is a dense network.
```

2. Network Visualization

2.1. Energy Function

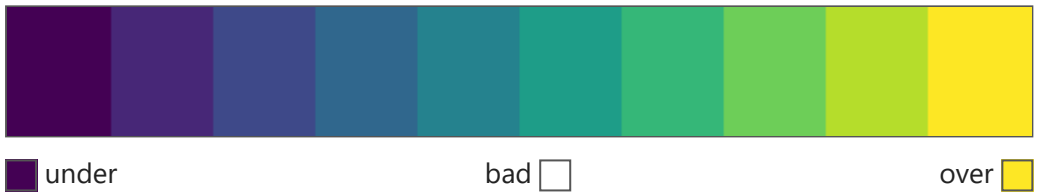
2.1.1. Davidson-Harel layout

For displaying the networks, the next color gradient is going to be used to represent the weights. Also, more important vertex are displayed bigger, and the edges with weights less than 2, are displayed in light gray in order to avoid a more crowded plot. The first plot is an example of the Davidson-Harel layout without the visual improvements mentioned before. It is used just as a reference to compare with the next plots.

In [19]:
```
cmap = cm.get_cmap('viridis', 10)
cmap
```

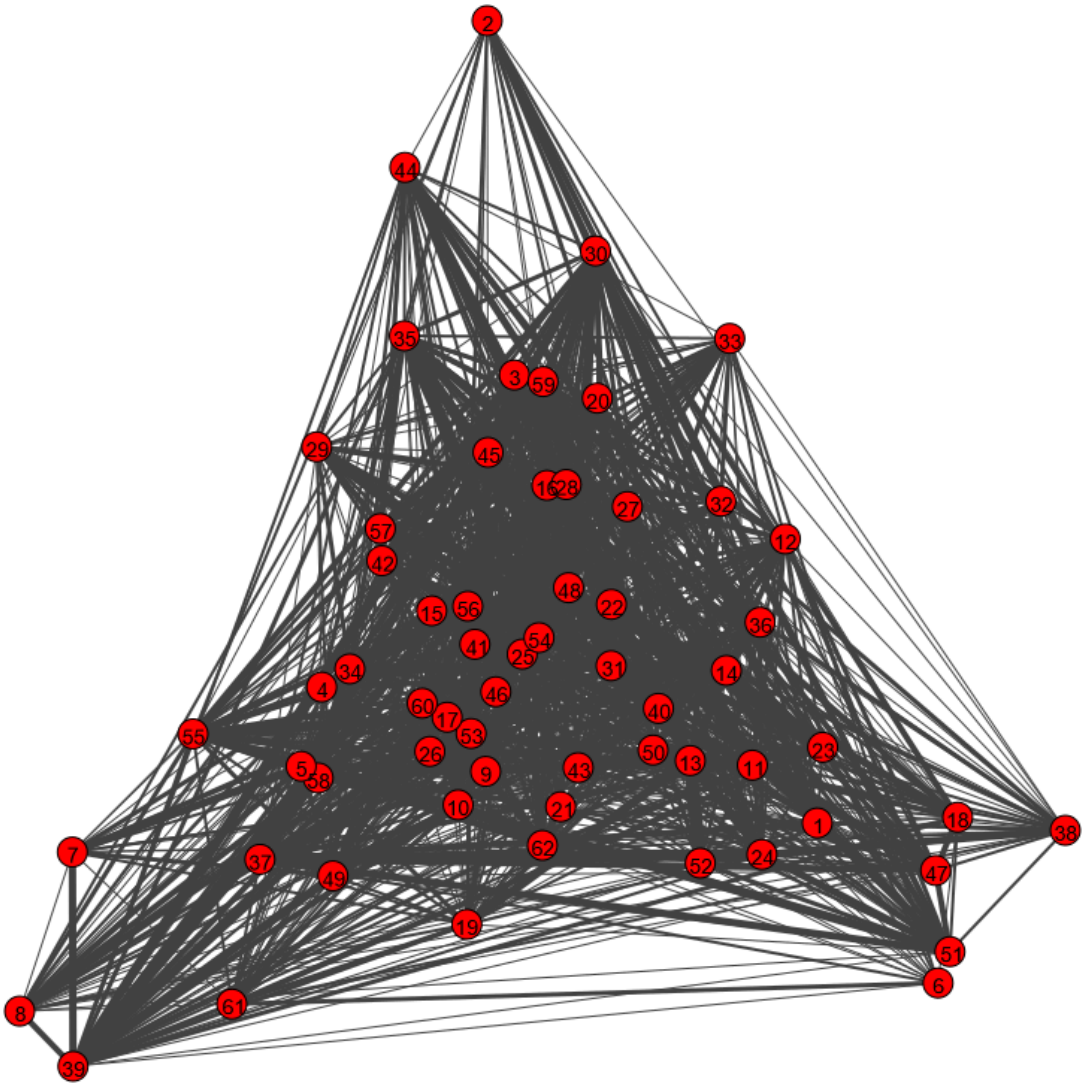
```
C:\Users\danie\AppData\Local\Temp\ipykernel_19860\3613669891.py:1: MatplotlibDepre
cationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be
removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_
cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis', 10)
```

Out[19]: **viridis**

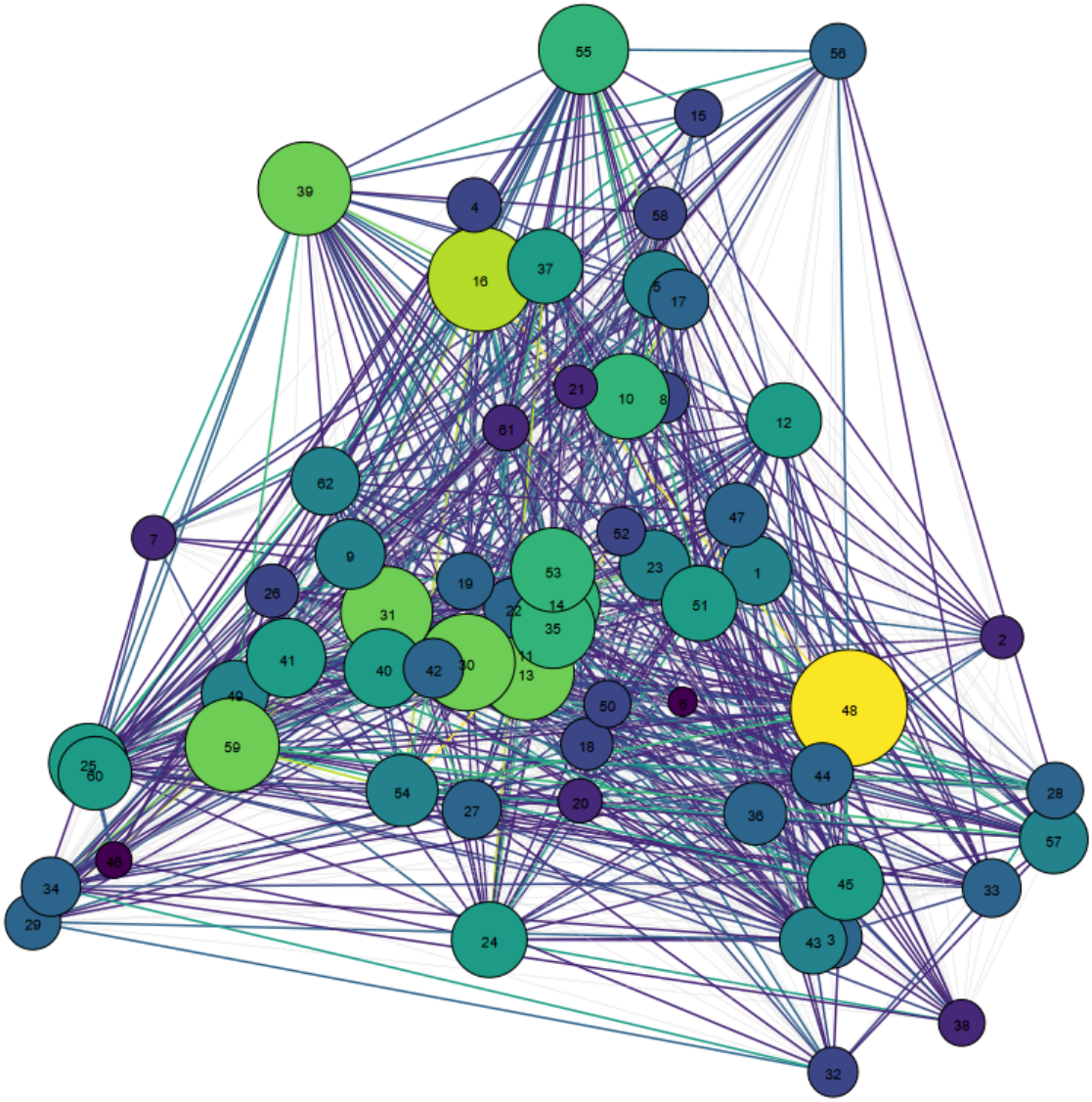


In [20]:
```
filename = "./imgs/layout_davidson_harel_simple.png"
plot_simple_graph(g, layout_type="davidson_harel", target=filename, display_img=Tru

filename = "./imgs/layout_davidson_harel.png"
plot_weighted_graph(g, layout_type="davidson_harel", target=filename, display_img=T
```

```
[✓] Skipping ./imgs/layout_davidson_harel_simple.png (already exists)
```



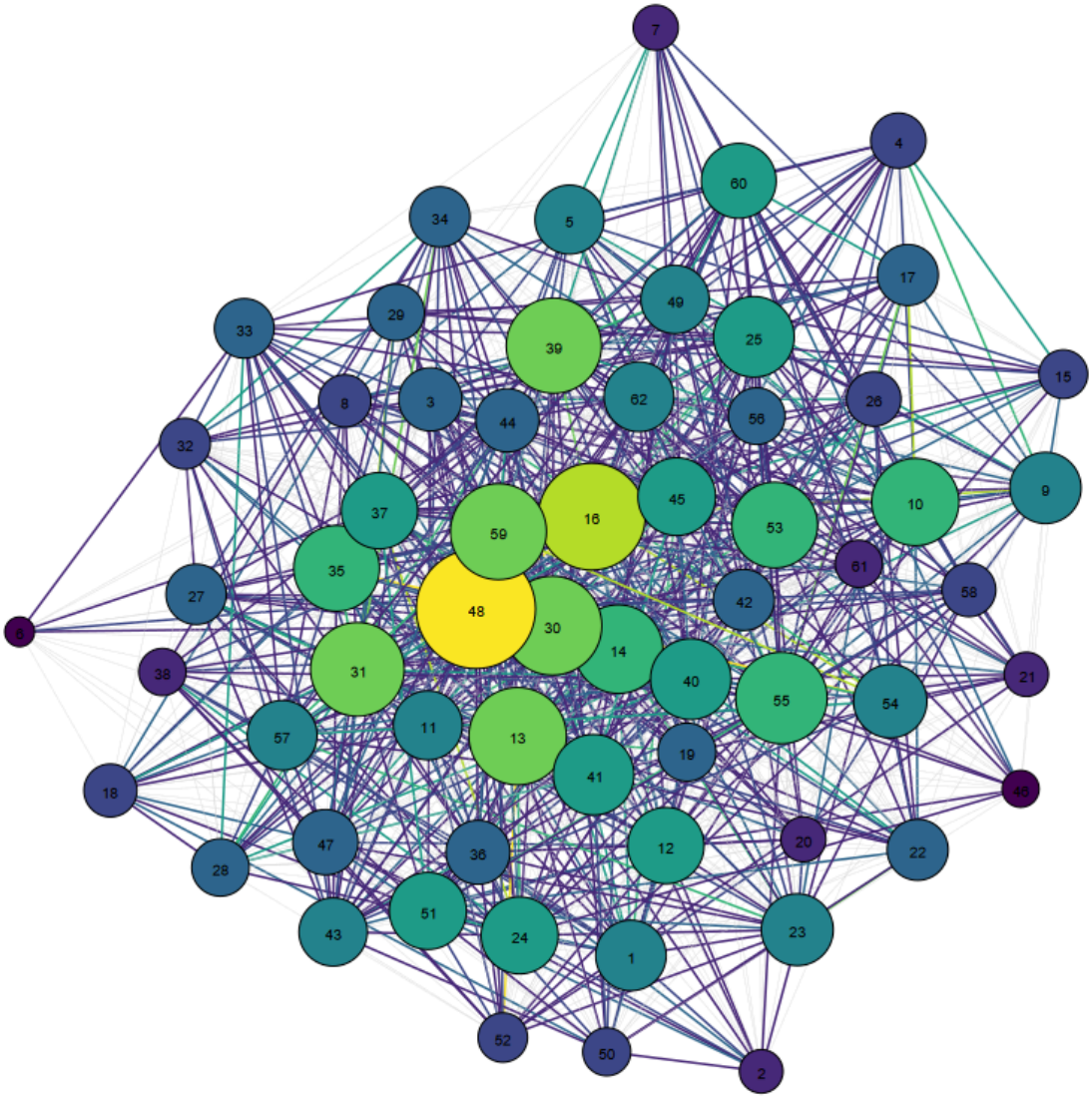
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)



2.1.2. Fruchterman-Reingold layout

In [21]:
```
filename = "./imgs/layout_fr.png"
plot_weighted_graph(g, layout_type="fr", target=filename, display_img=True)
```

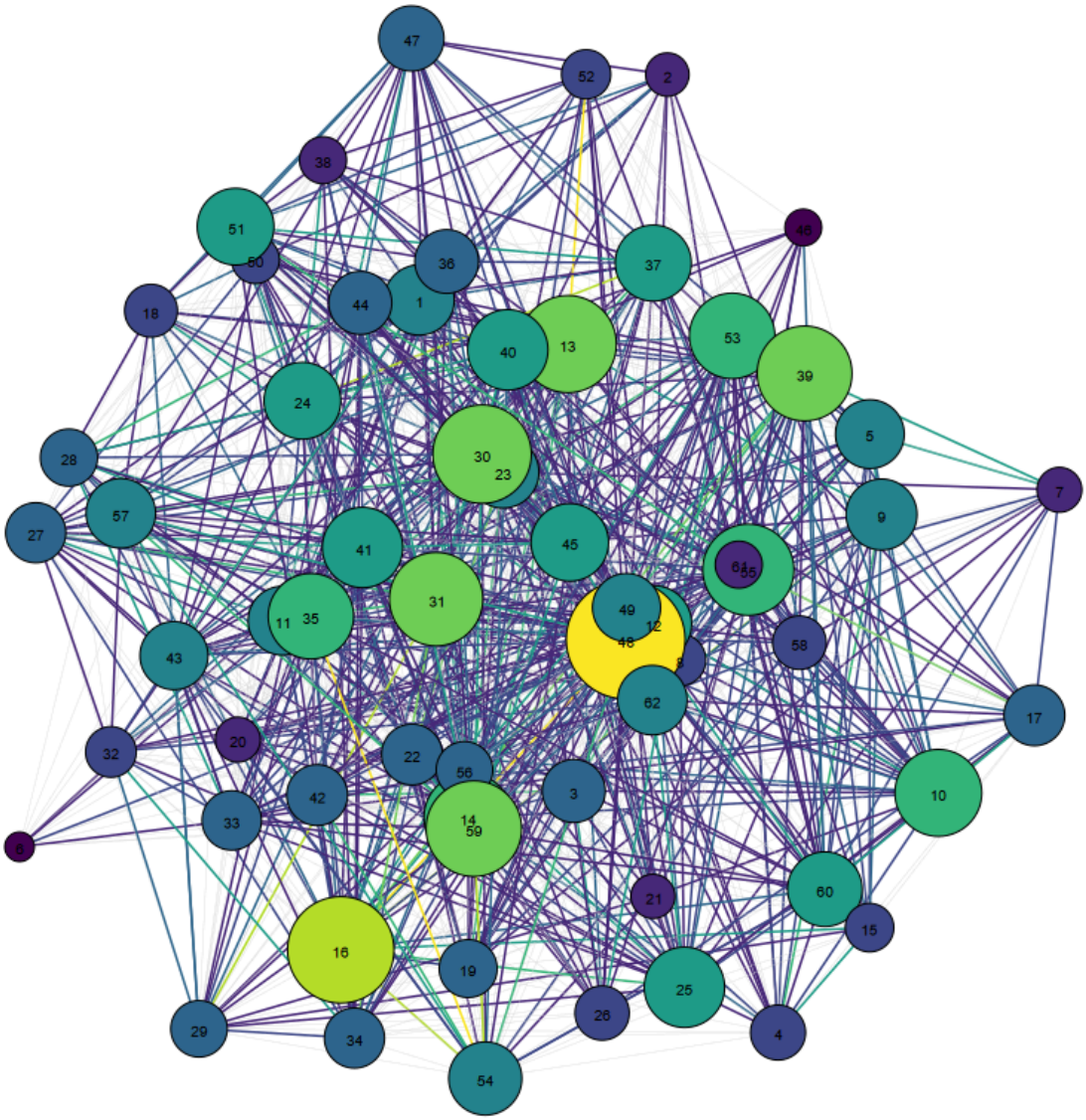
```
[✓] Skipping ./imgs/layout_fr.png (already exists)
```



2.1.3. Graphopt layout

In [22]:
```
filename = "./imgs/layout_graphopt.png"
plot_weighted_graph(g, layout_type="graphopt", target=filename, display_img=True)
```

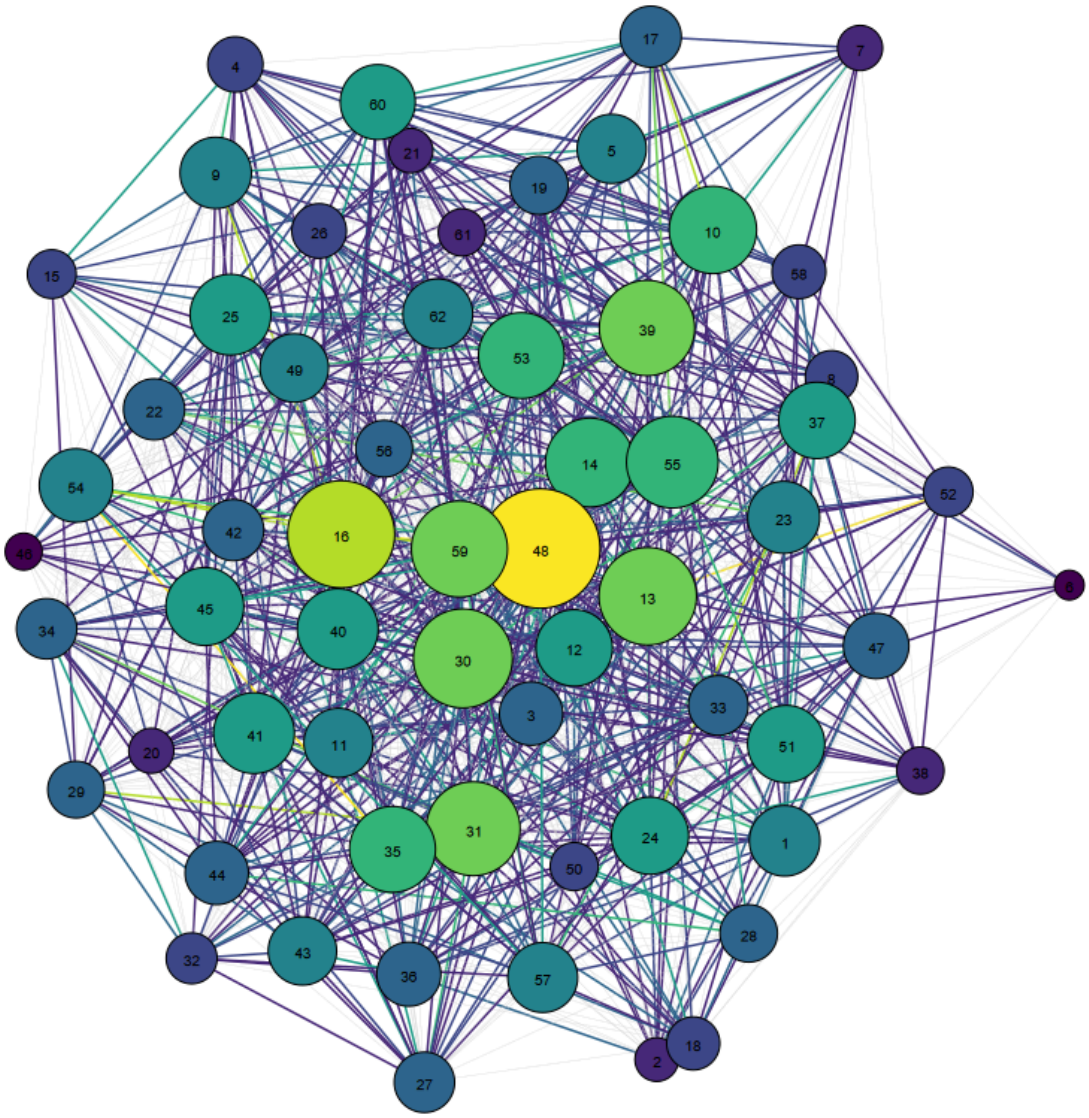
```
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
```



2.1.4. Kamada-Kawai layout

In [23]:
```
filename = "./imgs/layout_kk.png"
plot_weighted_graph(g, layout_type="kk", target=filename, display_img=True)
```

```
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



Comparison

Although we can not see much in the graph, some insights can be obtained:

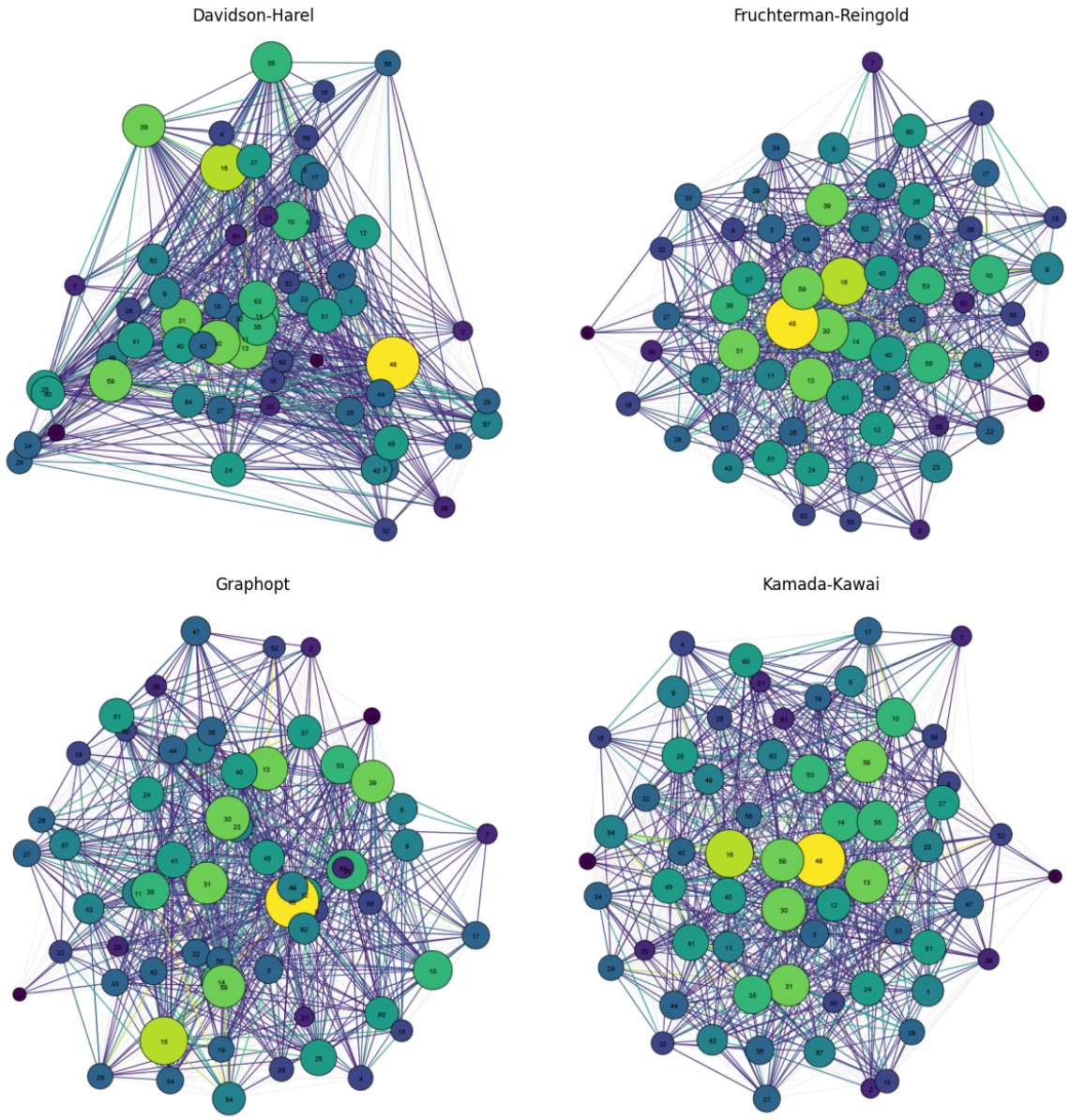
- Macaque 6 seems to have a great separation from the rest of the network in most of the layouts, except with the Davidson-Harel layout.
- Graphopt, Fruchterman-Reingold and Kamada-Kawai display a more circular network, leaving the less important vertex on the outskirts of the plot, while the other has a triangular shape separating groups of vertices, and mixing vertices of all importances in the center.
- The macaque with most importance (48) is displayed in the center with most of the layouts.

In general, all the layouts gave similar representations of the graph, except for Davidson-Harel.

In [24]:
```
import matplotlib.pyplot as plt
from PIL import Image
```

```python
# Layouts and labels
layouts = ["davidson_harel", "fr", "graphopt", "kk"]
layout_titles = ["Davidson-Harel", "Fruchterman-Reingold", "Graphopt", "Kamada-Kawa

# Save image paths
img_paths = []

# Generate and save plots using your updated function
for layout in layouts:
    filename = f"./imgs/layout_{layout}.png"
    plot_weighted_graph(g, layout_type=layout, target=filename, display_img=False)
    img_paths.append(filename)

# Display them in a 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 12))

for ax, img_path, title in zip(axs.flat, img_paths, layout_titles):
    img = Image.open(img_path)
    ax.imshow(img)
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
[✓] Skipping ./imgs/layout_davidson_harel.png (already exists)
[✓] Skipping ./imgs/layout_fr.png (already exists)
[✓] Skipping ./imgs/layout_graphopt.png (already exists)
[✓] Skipping ./imgs/layout_kk.png (already exists)
```



2.2. Multidimensional Scaling (MDS)

We create a 2D representation of the graph throught the MDS, where the matrix of distances are the shortest paths taking into account the inverse weights.

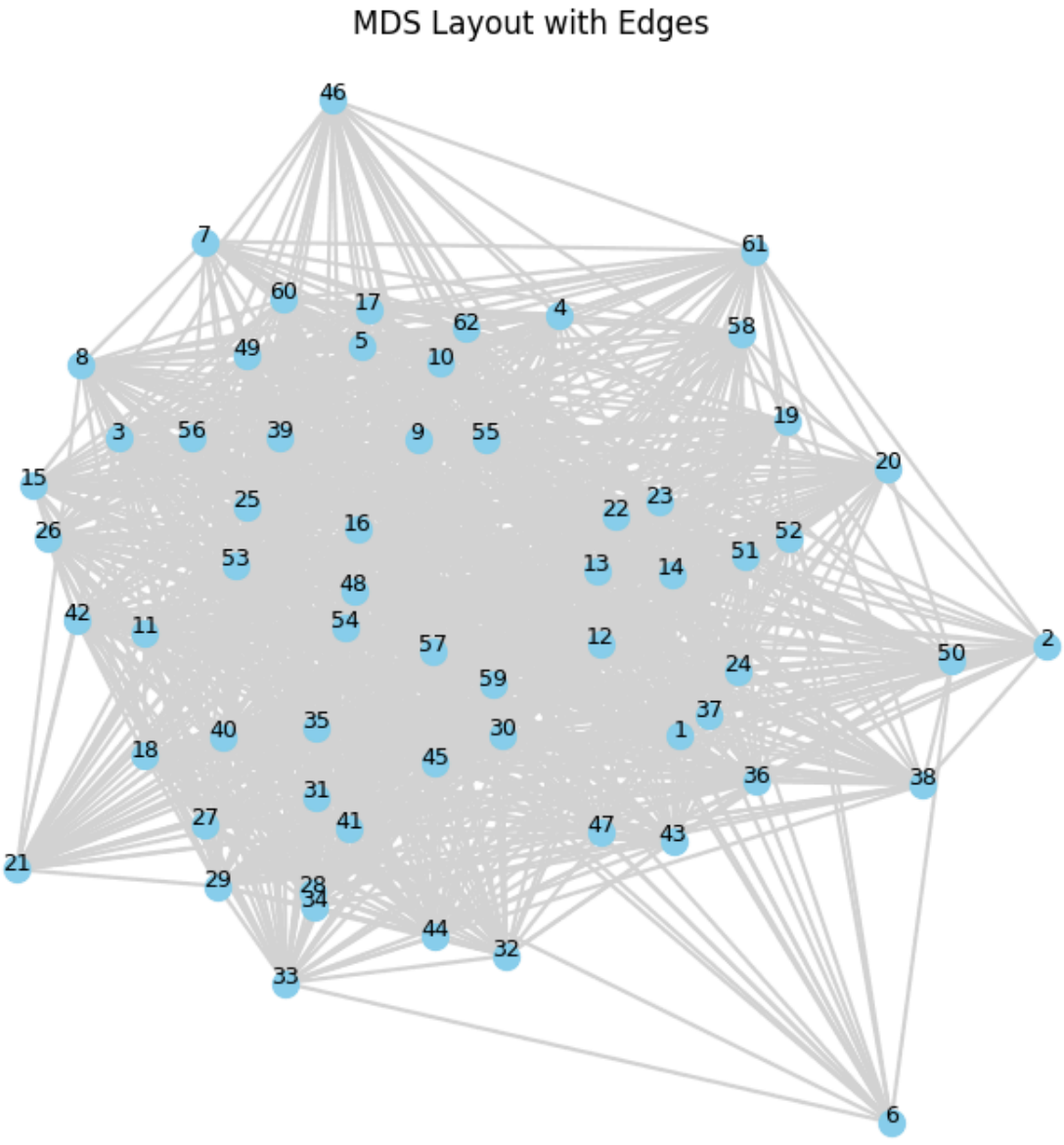
In [25]:
```
# Compute the shortest path distance matrix
dist_matrix = g.shortest_paths(weights="inv_weight")

# Run MDS
mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42)
mds_coords = mds.fit_transform(dist_matrix)
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_19860\2574702885.py:2: DeprecationWarn
ing: Graph.shortest_paths() is deprecated; use Graph.distances() instead
  dist_matrix = g.shortest_paths(weights="inv_weight")
```

Initially, we made a plot without displaying the weights. The result does not seem to be more informative than what we saw before, and the plot is still chaotic, giving not a lot of information.

In [26]:
```
plt.figure(figsize=(8, 8))
```

```
# Plot edges
for edge in g.get_edgelist():
    x0, y0 = mds_coords[edge[0]]
    x1, y1 = mds_coords[edge[1]]
    plt.plot([x0, x1], [y0, y1], color='lightgray', zorder=1)

# Plot nodes
plt.scatter(mds_coords[:, 0], mds_coords[:, 1], s=100, color='skyblue', zorder=2)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Edges")
plt.axis('off')
plt.show()
```



Then, we applied the color and size gradient as before, and we can see that the Multidimensional Scaling is by far the best representation that we have obtained of the network.

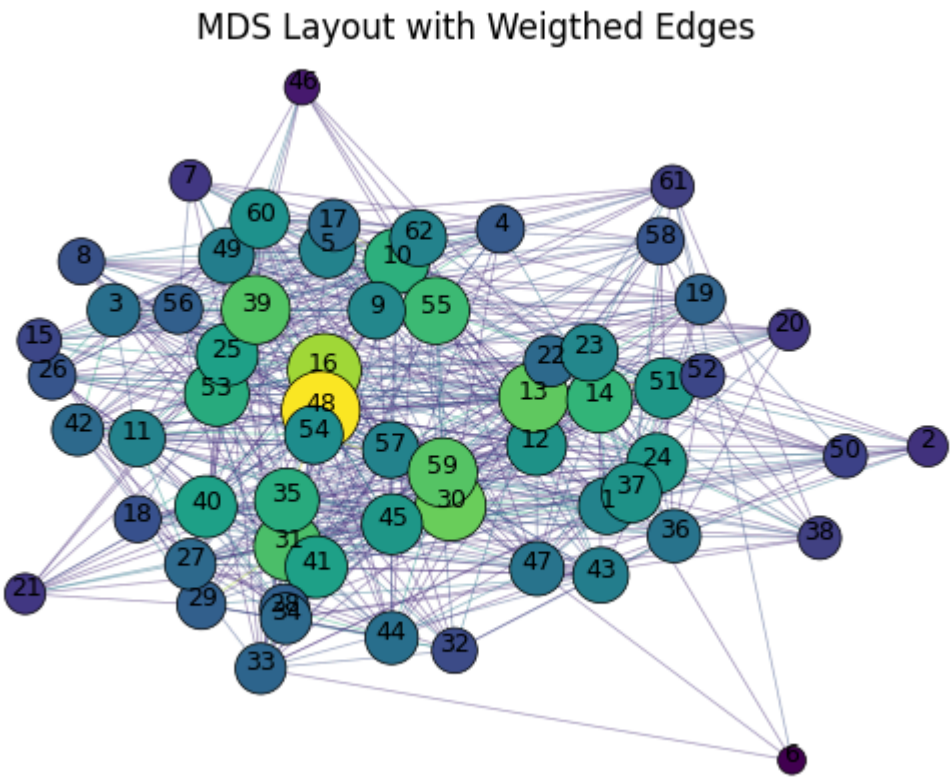
The most important vertex (Macaque 48) is in some way centered, and around it we can diferenciate two "rings". Closer to this node, the following most important macaques, and on

the outskirts of the plot, the less important ones. Allowing to see a kind of clustering based on the dominance-relations.

In [27]:
```
weights = np.array(g.es["weight"])
inv_weights = np.array(g.es["inv_weight"])
threshold = np.percentile(weights, 60)
norm = colors.Normalize(vmin=weights.min(), vmax=weights.max())
cmap = cm.get_cmap('viridis')
print("threshold: ", threshold)
edges_to_plot = [e.index for e in g.es if e["weight"] >= threshold]

# Compute node strength from strong edges
node_strength = np.zeros(g.vcount())
for e in g.es:
    if e["weight"] >= threshold:
        node_strength[e.source] += e["weight"]
        node_strength[e.target] += e["weight"]

# Normalize node strength to size and color
min_size, max_size = 100, 800
normalized_strength = (node_strength - node_strength.min()) / (node_strength.max()
node_sizes = min_size + (max_size - min_size) * normalized_strength
node_colors = [cmap(val) for val in normalized_strength]

for e in g.es:
    if e.index not in edges_to_plot:
        continue  # Skip plotting this weak edge
    i, j = e.source, e.target
    x0, y0 = mds_coords[i]
    x1, y1 = mds_coords[j]
    w = e["weight"]
    color = cmap(norm(w))
    plt.plot([x0, x1], [y0, y1], color=color, linewidth=0.5, alpha=0.5, zorder=1)

# Plot nodes
plt.scatter(
    mds_coords[:, 0], mds_coords[:, 1],
    s=node_sizes, color=node_colors,
    edgecolor='black', linewidth=0.5, zorder=2
)

# Add labels
for i, label in enumerate(g.vs["name"]):
    plt.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center', zo

plt.title("MDS Layout with Weigthed Edges")
plt.axis('off')
plt.show()
```

```
threshold:  2.0
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_19860\3662256221.py:5: MatplotlibDepre
cationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be
removed in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_
cmap()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap('viridis')
```



In [28]:
```
#get_node_info(g, 6, threshold_pt=60)
```

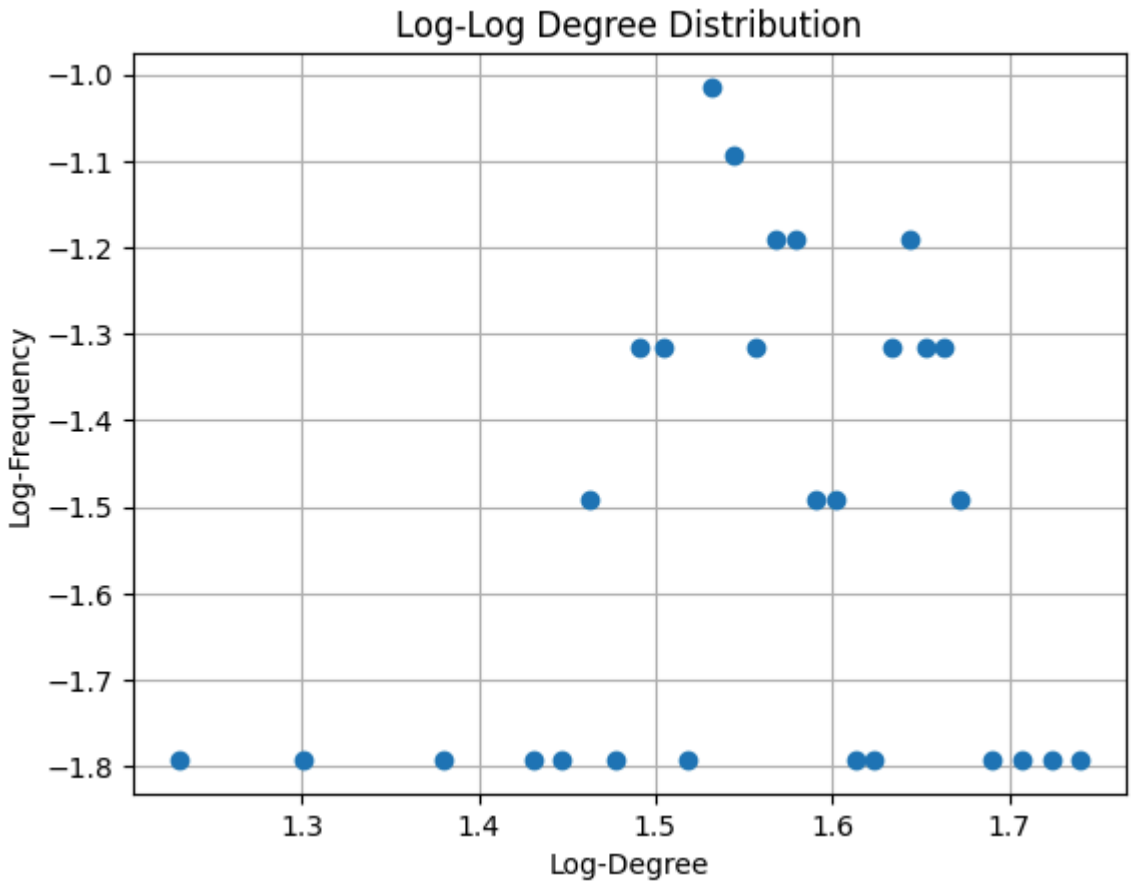
3. Vertex Characteristics

3.1. Degree Distribution

3.1.1. Unweighted log-log Degree Distribution

We clearly see that the degree distribution is not following a power law. Just for further proof, we will fit the data to a power law distribution and validate that the fit is not good.

In [29]:
```
degrees = g.degree()
unique_degrees, counts = np.unique(degrees, return_counts=True)

# Log-Log degree distribution
log_degrees = np.log10(unique_degrees[probs > 0])
log_probs = np.log10(probs[probs > 0])

plt.figure()
plt.plot(log_degrees, log_probs, 'o')
plt.xlabel("Log-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Degree Distribution")
plt.grid(True)
plt.show()
```



In [30]:
```
# Clean up for fitting: remove zero-degree entries to avoid log(0)
mask = (unique_degrees > 0) & (probs > 0)
x = unique_degrees[mask]
y = probs[mask]

log_x = np.log10(x)
log_y = np.log10(y)

# Use weights = 1/x to mimic WLS emphasis on lower degrees
X = sm.add_constant(log_x)
wls_model = sm.WLS(log_y, X, weights=1/log_x)
results = wls_model.fit()

print(results.summary())
alpha = results.params[1]
print(f"\nEstimated alpha: {alpha:.4f}")
```

```
                            WLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.049
Model:                            WLS   Adj. R-squared:                  0.013
Method:                 Least Squares   F-statistic:                     1.353
Date:                Mon, 12 May 2025   Prob (F-statistic):              0.255
Time:                        20:27:36   Log-Likelihood:                -2.0220
No. Observations:                  28   AIC:                             8.044
Df Residuals:                      26   BIC:                             10.71
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.2605      0.626     -3.610      0.001      -3.547      -0.973
x1             0.4694      0.404      1.163      0.255      -0.360       1.299
==============================================================================
Omnibus:                        5.630   Durbin-Watson:                   0.997
Prob(Omnibus):                  0.060   Jarque-Bera (JB):                2.134
Skew:                           0.299   Prob(JB):                        0.344
Kurtosis:                       1.787   Cond. No.                         27.0
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly s
pecified.

Estimated alpha: 0.4694
```

As we can see, the R-squared is 0.0494 and the adjusted R-squared is 0.0129, which means that the model does not fit the data well.

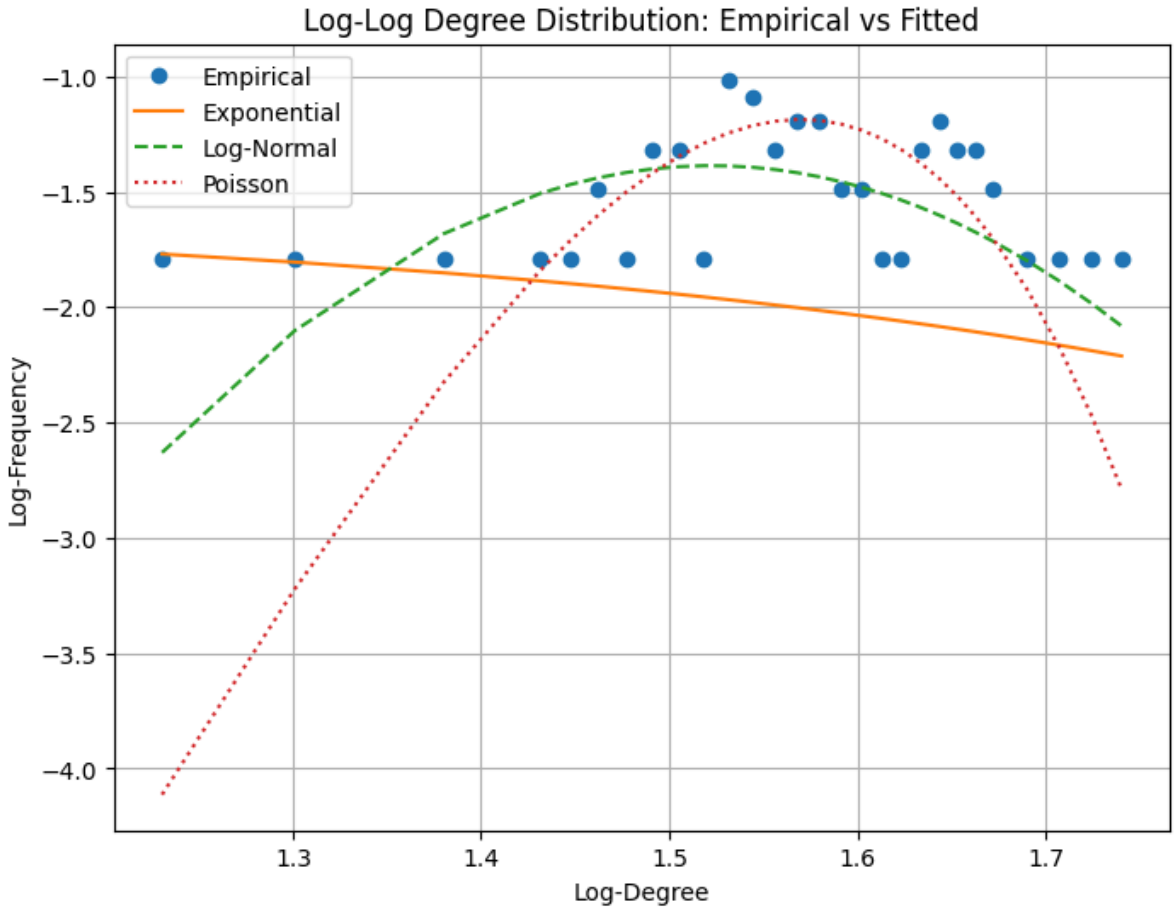
In [31]:
```python
r2 = results.rsquared
print(f"R-squared: {r2:.4f}")
adj_r2 = results.rsquared_adj
print(f"Adjusted R-squared: {adj_r2:.4f}")
```

```
R-squared: 0.0494
Adjusted R-squared: 0.0129
```

We tried to plot the data on other distributions, but the results were not good either.

In [32]:
```python
# Exponential
exp_loc, exp_scale = expon.fit(x, floc=0)
pdf_exp = expon.pdf(x, loc=exp_loc, scale=exp_scale)

# Log-Normal
ln_shape, ln_loc, ln_scale = lognorm.fit(x, floc=0)
pdf_ln = lognorm.pdf(x, ln_shape, loc=ln_loc, scale=ln_scale)

# Poisson (discrete, use PMF)
mu = np.mean(degrees)
pmf_pois = poisson.pmf(x, mu)

# Step 3: Log-transform fitted values
log_pdf_exp = np.log10(pdf_exp)
log_pdf_ln = np.log10(pdf_ln)
log_pmf_pois = np.log10(pmf_pois)

# Step 4: Plot
plt.figure(figsize=(8, 6))
plt.plot(log_x, log_y, 'o', label="Empirical")
plt.plot(log_x, log_pdf_exp, '-', label="Exponential")
```

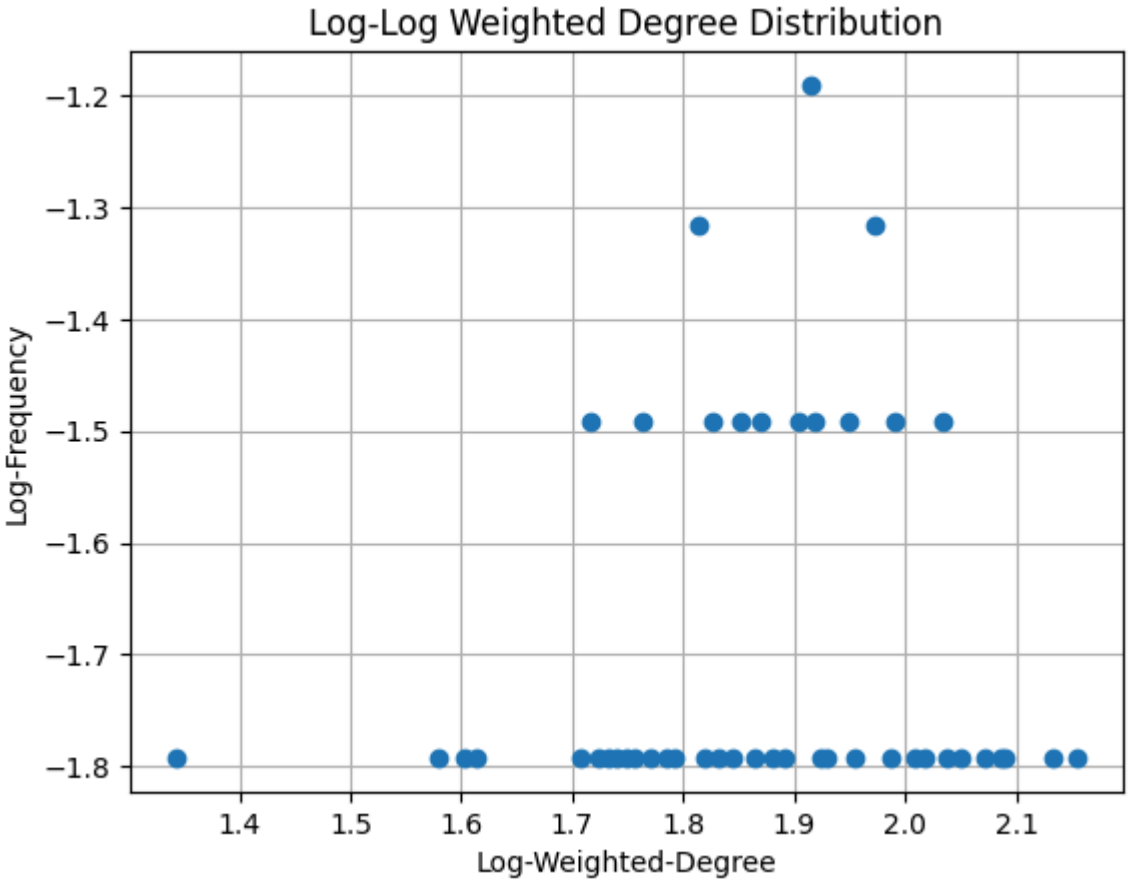
```
plt.plot(log_x, log_pdf_ln, '--', label="Log-Normal")
plt.plot(log_x, log_pmf_pois, ':', label="Poisson")
plt.xlabel("Log-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Degree Distribution: Empirical vs Fitted")
plt.legend()
plt.grid(True)
plt.show()
```



3.1.2. Weighted log-log Degree Distribution

We can see that the weighted degree distribution is not following a power law either. Just by looking at the plot we can see that the data is not following a straight line and the shape is similar to the unweighted one, so we won't compute the fit for this one.

In [33]:
```
weighted_degrees = g.strength(weights="weight")
unique_w_deg, w_counts = np.unique(weighted_degrees, return_counts=True)
w_probs = w_counts / sum(w_counts)

# Log-log weighted degree distribution
log_w_deg = np.log10(unique_w_deg[w_probs > 0])
log_w_probs = np.log10(w_probs[w_probs > 0])

plt.figure()
plt.plot(log_w_deg, log_w_probs, 'o')
plt.xlabel("Log-Weighted-Degree")
plt.ylabel("Log-Frequency")
plt.title("Log-Log Weighted Degree Distribution")
plt.grid(True)
plt.show()
```



3.2. Centrality Measures

In [34]:
```python
def top_k_centrality(centrality, label, k=5):
    top_indices = np.argsort(centrality)[::-1][:k]
    print(f"\n 🔹 Top {k} nodes by {label}:")
    for idx in top_indices:
        print(f"{g.vs[idx]['name']}: {centrality[idx]:.4f}")
```

3.2.1. Degree Centrality

The most central vertices according to the degree centrality are the ones with most connections. On the weighted case, the most central is the one with most connections, taking into account the weights. In both cases, the most central vertex is the 48. We can check that in the previous section, on the graphs plots, the top vertices are the same. (As their size and color in the plot depend on the weights)

In [35]:
```python
# Unweighted degree
degrees = np.array(g.degree())
top_unweighted = np.argsort(degrees)[::-1]  # descending order


# Weighted degree (strength)
weighted_degrees = np.array(g.strength(weights="weight"))
top_weighted = np.argsort(weighted_degrees)[::-1]

# How many top nodes to show
k = 5

print(" 🔹 Top vertices by unweighted degree:")
for idx in top_unweighted[:k]:
    print(f"{g.vs[idx]['name']}: degree = {degrees[idx]}")
```

```
print("\n 🔹 Top vertices by weighted degree:")
for idx in top_weighted[:k]:
    print(f"{g.vs[idx]['name']}: weighted degree = {weighted_degrees[idx]}")
```

```
 🔹 Top vertices by unweighted degree:
48: degree = 55
30: degree = 53
59: degree = 51
12: degree = 49
40: degree = 47

 🔹 Top vertices by weighted degree:
48: weighted degree = 143.0
16: weighted degree = 136.0
59: weighted degree = 123.0
30: weighted degree = 122.0
13: weighted degree = 118.0
```

As we can see on the degree distribution plotted at the beginning, the most connected vertices have a significantly higher degree than the rest of the vertices. This means that they are more important in the network, and that they are more likely to be connected to other important vertices. We can check that by computing the z-scores of the top vertices. The z-score is a measure of how many standard deviations a value is from the mean. A high z-score means that the vertex is more important than the average vertex in the network.

In our case, the top 5 nodes by unweighted degree have z-scores ranging from 1.26 to 2.34, with the highest degree node reaching 2.34 standard deviations above the mean. These values suggest that while the top nodes are more central than average, they are not extreme outliers. The z-scores are moderately high, indicating a moderate centrality hierarchy but not a strong dominance by a few nodes. There's a gradient of centrality rather than a sharp concentration.

For the weighted degree (strength), the top 5 nodes show z-scores between 1.64 and 2.67, which are consistently higher than those of the unweighted case. The top node, with a z-score of 2.67, is particularly notable — this suggests it is substantially more central in terms of accumulated edge weight. Overall, weighted centrality shows a more pronounced disparity, implying that some nodes are involved in stronger or more frequent interactions than others, even if the number of neighbors isn't dramatically different.

In [36]:
```
degrees = np.array(g.degree())
top_k = 5
sorted_deg = np.sort(degrees)[::-1]
top = sorted_deg[:top_k]

mean_deg = degrees.mean()
std_deg = degrees.std()
print("&"*50)
print("Top degrees:", top)
print(f"Mean: {mean_deg:.2f}, Std: {std_deg:.2f}")
z_scores = (top - mean_deg) / std_deg
print("Z-scores of top nodes:", z_scores)
print("&"*50)

weighted_degrees = np.array(g.strength(weights="weight"))
sorted_weighted_deg = np.sort(weighted_degrees)[::-1]
top_weighted = sorted_weighted_deg[:top_k]
```

```
mean_weighted_deg = weighted_degrees.mean()
std_weighted_deg = weighted_degrees.std()
print("\n")
print("&"*50)
print("Top weighted degrees:", top_weighted)
print(f"Mean: {mean_weighted_deg:.2f}, Std: {std_weighted_deg:.2f}")
z_scores_weighted = (top_weighted - mean_weighted_deg) / std_weighted_deg
print("Z-scores of top nodes (weighted):", z_scores_weighted)
print("&"*50)
```

```
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
Top degrees: [55 53 51 49 47]
Mean: 37.65, Std: 7.40
Z-scores of top nodes: [2.34487777 2.07465022 1.80442267 1.53419512 1.26396757]
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&


&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
Top weighted degrees: [143. 136. 123. 122. 118.]
Mean: 78.55, Std: 24.11
Z-scores of top nodes (weighted): [2.67363053 2.38325124 1.84397541 1.80249266 1.6
3656163]
&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&&
```

3.2.2. Closeness Centrality

The closeness centrality is a measure of how close a vertex is to all other vertices in the network. The most central vertex is the one that can reach all other vertices in the shortest time.

In the unweighted case, node 48 is the most central, suggesting it has short paths to many other macaques in terms of connection steps. However, when accounting for the number of dominance interactions (i.e., using inverse interaction frequency as distance), node 16 becomes the most central. This indicates that node 16, and others like 54 and 13, are not just well connected, they are connected through frequent dominance interactions, placing them strategically at the core of high-intensity interaction clusters.

In [37]:
```
# --- Closeness centrality ---
closeness_unw = np.array(g.closeness(normalized=True))
closeness_w = np.array(g.closeness(weights="inv_weight", normalized=True))

top_k_centrality(closeness_unw, "Closeness (unweighted)")
top_k_centrality(closeness_w, "Closeness (weighted)")
```

```
🔹 Top 5 nodes by Closeness (unweighted):
48: 0.9104
30: 0.8841
59: 0.8592
12: 0.8356
40: 0.8133

🔹 Top 5 nodes by Closeness (weighted):
16: 2.7633
54: 2.5946
59: 2.5703
48: 2.5627
13: 2.4766
```

3.2.3. Betweenness Centrality

Betweenness centrality is a measure of how often a vertex acts as a bridge along the shortest path between two other vertices. The most central vertex is the one that lies on the most shortest paths between other vertices.

Node 48 also leads in unweighted betweenness, suggesting it may act as a bridge between groups in the macaque social structure. However, in the weighted analysis, node 16 dominates. This means node 16 frequently lies along the shortest, most interaction-rich paths between other individuals.

In a social dominance context, this could point to central individuals involved in resolving or mediating dominance dynamics, possibly respected or high-status macaques based on the frequency and breadth of their dominance interactions.

In [38]:
```
n = g.vcount()

# --- Betweenness centrality ---
betweenness_unw = np.array(g.betweenness())
betweenness_unw = betweenness_unw / ((n - 1) * (n - 2) / 2)

betweenness_w = np.array(g.betweenness(weights="inv_weight"))
betweenness_w = betweenness_w / ((n - 1) * (n - 2) / 2)

top_k_centrality(betweenness_unw, "Betweenness (unweighted)")
top_k_centrality(betweenness_w, "Betweenness (weighted)")
```

```
🔹 Top 5 nodes by Betweenness (unweighted):
48: 0.0166
30: 0.0139
59: 0.0126
12: 0.0119
55: 0.0112

🔹 Top 5 nodes by Betweenness (weighted):
16: 0.1229
13: 0.0839
48: 0.0615
59: 0.0607
54: 0.0552
```

3.2.4. Eigenvector Centrality

Eigenvector centrality is a measure of the influence of a node in a network. It considers not just the number of connections a node has, but also the quality and influence of those connections. A node is considered important if it is connected to other important nodes.

In the unweighted case, node 48 stands out as the most "influential" macaque, followed closely by nodes 30 and 59, likely forming a core subgroup in the dominance network.

When accounting for interaction strength (weighted), node 16 surpasses 48, suggesting that 16 may not only be connected to central individuals, but those connections are reinforced by frequent dominance interactions.

This shift highlights how interaction intensity reveals hidden social influence, not captured by topology alone.

In [39]:
```
# --- Eigenvector centrality ---
eigen_unw = np.array(g.eigenvector_centrality())
eigen_w = np.array(g.eigenvector_centrality(weights="weight"))
top_k_centrality(eigen_unw, "Eigenvector (unweighted)")
top_k_centrality(eigen_w, "Eigenvector (weighted)")
```

```
🔹 Top 5 nodes by Eigenvector (unweighted):
48: 1.0000
30: 0.9768
59: 0.9403
12: 0.9002
16: 0.8801

🔹 Top 5 nodes by Eigenvector (weighted):
16: 1.0000
48: 0.9999
59: 0.8540
30: 0.8478
13: 0.8069
```

3.2.5. PageRank centrality

PageRank centrality is a measure of the importance of a node in a network, based on the idea that important nodes are likely to be connected to other important nodes.

Node 48 consistently ranks highest in both weighted and unweighted PageRank, confirming its prominent role in the dominance network. With weights included, node 16 also emerges near the top, reinforcing the idea that frequency of interactions elevates its perceived social standing.

The alignment between PageRank and eigenvector results supports a stable, central core group, with node 48 as a persistent central figure and node 16 gaining importance when intensity is factored in.

In [40]:
```
# --- PageRank ---
pagerank_unw = np.array(g.pagerank())
pagerank_w = np.array(g.pagerank(weights="weight"))

top_k_centrality(pagerank_unw, "PageRank (unweighted)")
top_k_centrality(pagerank_w, "PageRank (weighted)")
```

```
🔹 Top 5 nodes by PageRank (unweighted):
48: 0.0226
30: 0.0218
59: 0.0210
12: 0.0203
40: 0.0196

🔹 Top 5 nodes by PageRank (weighted):
48: 0.0274
16: 0.0259
59: 0.0239
30: 0.0238
13: 0.0232
```

4. Edge Characteristics

In [41]:
```
def top_k_edges(edge_btw, label, k=5):
    top_indices = np.argsort(edge_btw)[::-1][:k]
```

```
    print(f"\n 🔹 Top {k} edges by {label}:")
    for idx in top_indices:
        e = g.es[idx]
        v1, v2 = g.vs[e.source]["name"], g.vs[e.target]["name"]
        print(f"{v1} - {v2}: {edge_btw[idx]:.4f}")
```

4.1. Edge Betweenness Centrality

Edge betweenness centrality is a measure of how often an edge lies on the shortest path between two other vertices. The most central edge is the one that lies on the most shortest paths between other vertices.

The top edges (e.g., between nodes 48–6, 48–7, and 30–6) appear to serve as structural bridges in the macaque dominance network. These edges lie on many of the shortest paths and likely connect dense subgroups or clusters, making them essential for maintaining overall network connectivity.

Their high centrality suggests that dominance interactions between these individuals are critical for the indirect relational structure, even if the interactions aren't especially frequent.

When accounting for the number of interactions (using inv_weight as distance), a different set of edges emerges, notably 59–54, 16–48, and 35–54. These links carry significant interaction volume while still connecting crucial paths.

This indicates that not only do these edges bridge important network regions, but they also do so via frequent dominance interactions, which could point to key pathways in reinforcing or mediating social structure through repetition and intensity.

In [42]:
```
# Unweighted edge betweenness
edge_btw_unw = np.array(g.edge_betweenness())

# Weighted edge betweenness using inverse weight
edge_btw_w = np.array(g.edge_betweenness(weights="inv_weight"))

top_k_edges(edge_btw_unw, "Edge Betweenness (unweighted)")
top_k_edges(edge_btw_w, "Edge Betweenness (weighted)")
```

```
 🔹 Top 5 edges by Edge Betweenness (unweighted):
48 - 6: 4.7522
48 - 7: 4.6561
30 - 6: 4.6432
61 - 6: 4.2829
12 - 6: 4.0198

 🔹 Top 5 edges by Edge Betweenness (weighted):
59 - 54: 65.0000
16 - 48: 60.0000
35 - 54: 58.0000
16 - 9: 57.5000
16 - 54: 54.0000
```

While nodes 16 and 48 consistently appear as the most central individuals across multiple centrality measures, we cannot conclude whether they are dominant or subordinate, since the network is undirected and lacks information about interaction direction. Their high centrality indicates they are frequent and structurally important participants in dominance interactions, and the edge connecting them likely represents a key social relationship,

potentially reflecting a complex or contested dominance dynamic between two influential individuals.

5. Network Cohesion

In [43]:
```
def plot_mds_graph(g, mds_coords, color_vector, color_labels=None,
                   color_title="Legend", color_list=None, subtitle=None,
                   highlight_edges=None, highlight_color='red', highlight_width=2.0
    """
    Plots a graph using given MDS coordinates, with optional edge highlighting.

    Parameters:
        g              : igraph.Graph object
        mds_coords     : 2D array of shape (n_nodes, 2) with MDS coordinates
        color_vector   : list or array of values for node colors (e.g., k-core, com
        color_labels   : optional list of legend labels (defaults to unique values)
        color_title    : title for the legend
        color_list     : optional list of colors to use (defaults to preset)
        highlight_edges: list of edges (as tuples) to highlight (default: None)
        highlight_color: color for highlighted edges (default: 'red')
        highlight_width: line width for highlighted edges (default: 2.0)
    """
    # 1. Normalize node strength for sizing
    strength = np.array(g.strength(weights=g.es["weight"]))
    normalized_strength = (strength - strength.min()) / (strength.max() - strength.
    node_sizes = 100 + 700 * normalized_strength

    # 2. Map values to discrete colors
    unique_vals = sorted(set(color_vector))
    color_list = color_list or ['blue', 'green', 'red', 'purple', 'yellow', 'orange
    cmap = ListedColormap(color_list[:len(unique_vals)])
    val_to_index = {val: i for i, val in enumerate(unique_vals)}
    color_indices = [val_to_index[val] for val in color_vector]

    # 3. Plot
    fig, ax = plt.subplots(figsize=(8, 6))

    # Draw all edges (lightgray)
    for edge in g.get_edgelist():
        x = [mds_coords[edge[0], 0], mds_coords[edge[1], 0]]
        y = [mds_coords[edge[0], 1], mds_coords[edge[1], 1]]
        ax.plot(x, y, color='lightgray', linewidth=0.8, zorder=1)

    # Draw highlighted edges (if provided)
    if highlight_edges:
        for edge in highlight_edges:
            x = [mds_coords[edge[0], 0], mds_coords[edge[1], 0]]
            y = [mds_coords[edge[0], 1], mds_coords[edge[1], 1]]
            ax.plot(x, y, color=highlight_color, linewidth=highlight_width, zorder=

    # Draw nodes
    scatter = ax.scatter(
        mds_coords[:, 0], mds_coords[:, 1], s=node_sizes, alpha=0.9,
        c=color_indices, cmap=cmap, edgecolor='k', zorder=2
    )

    # Add labels
    for i, label in enumerate(g.vs["name"]):
        ax.text(mds_coords[i, 0], mds_coords[i, 1], label, fontsize=9, ha='center',

    # Legend
```

```
    if color_labels is None:
        color_labels = unique_vals
    legend_handles = [
        Patch(color=cmap(i), label=str(label)) for i, label in enumerate(color_labe
    ]
    ax.legend(handles=legend_handles, title=color_title, bbox_to_anchor=(1.05, 1),

    ax.set_title("MDS Graph Representation", fontsize=14)
    if subtitle:
        ax.text(0.5, 0.97, subtitle, fontsize=11, ha='center', transform=ax.transAx

    ax.set_xlabel("MDS1")
    ax.set_ylabel("MDS2")
    plt.tight_layout()
    plt.show()
```

5.1. Local Density

Initially, we want to know if there is a common dense structure between small groups of vertices.

5.1.1. Cliques / Maximal Cliques

We study the number of complete subgraphs and its size in order to determine the magnitude of direct relations between the vertices.

- The biggest complete subgraph has 13 vertices. This are very dense groups where every vertex has 12 connections inside it.
- The most common size of cliques is 7, but most of them are contained in other subgraphs (only 482 are maximal cliques).
- All of the cliques of small size are contained in bigger cliques

In conclusion the network has high local density.

In [44]:
```
# All cliques
cliques = g.cliques()
clique_sizes = [len(c) for c in cliques]
clique_count = Counter(clique_sizes)

# Maximal cliques
max_cliques = g.maximal_cliques()
max_clique_sizes = [len(c) for c in max_cliques]
max_clique_count = Counter(max_clique_sizes)

# Combine into a table
all_lengths = sorted(set(clique_count) | set(max_clique_count))
df = pd.DataFrame({
    "Clique Size": all_lengths,
    "Number of Cliques": [clique_count.get(k, 0) for k in all_lengths],
    "Number of Maximal Cliques": [max_clique_count.get(k, 0) for k in all_lengths]
})

# Display the table
print(df.to_string(index=False))
```

Clique Size	Number of Cliques	Number of Maximal Cliques
1	62	0
2	1167	0
3	9781	0
4	43826	0
5	117120	15
6	200245	119
7	228365	482
8	176900	848
9	92795	1273
10	32168	1480
11	7022	1033
12	882	328
13	50	50

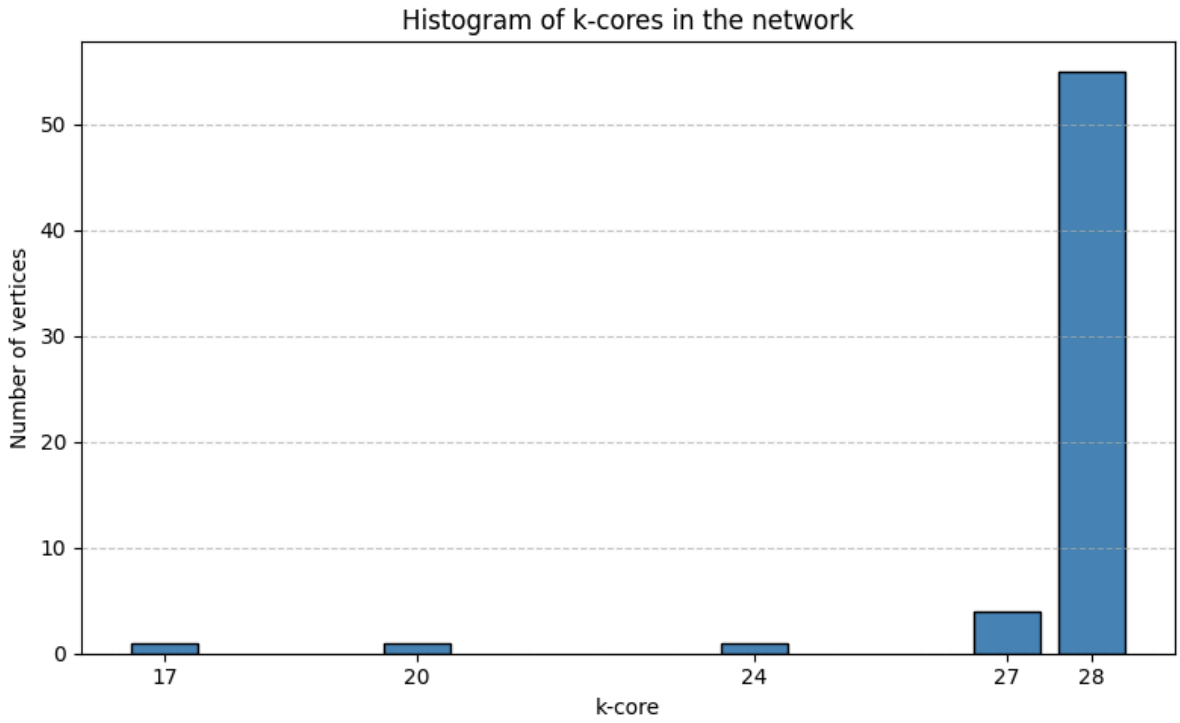
5.1.2. K-cores

A k-core is a subgraph for which all the vertex in it has at least degree iqual to k, that means, has at least k neighbors within it.

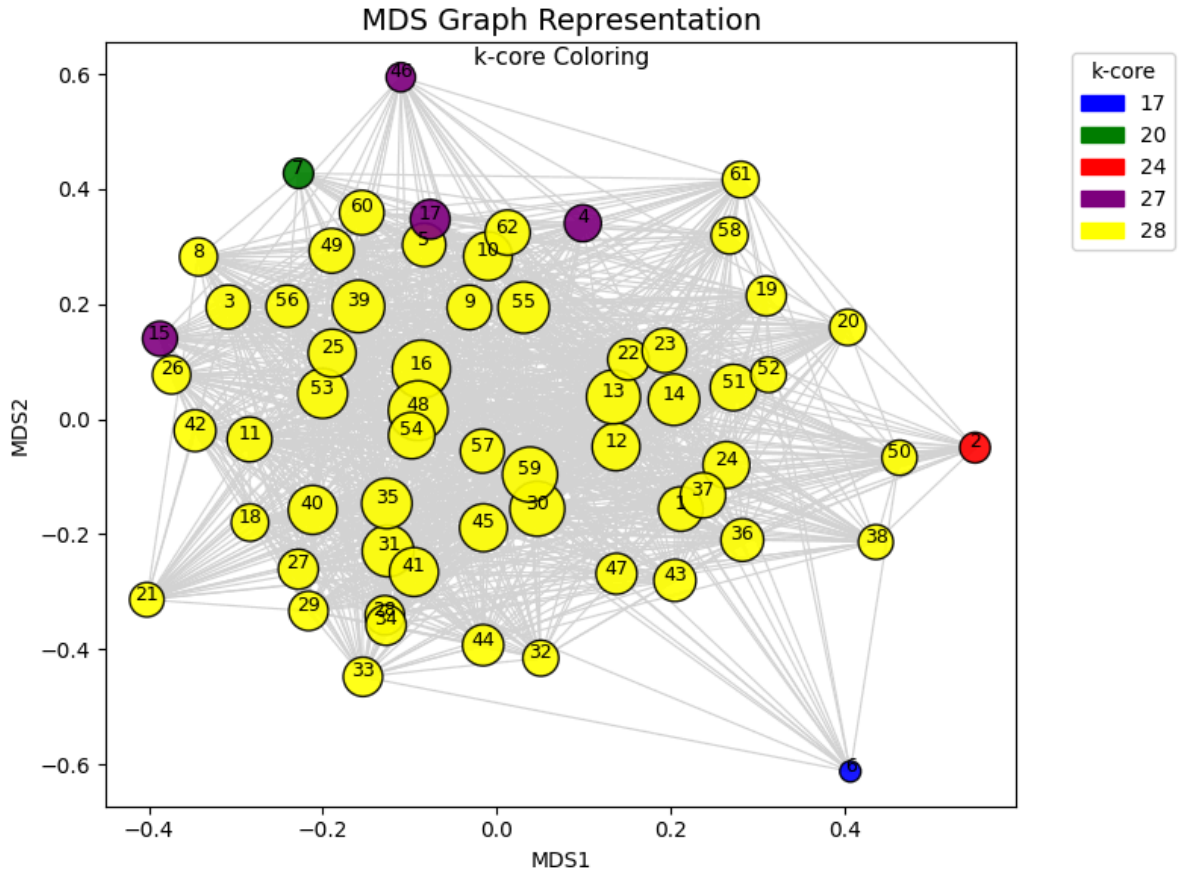
In our case, the biggest k-core (55 vertices) occurs in k=28, it suggest that the make part of the most tighly connect core. The others cores finded (k=17, 20, 24, 27) has less than 5 nodes each, representing less dense peripherial nodes.

With this two results we confirm that the network has both, strong local (cliques) and dense, cohesive global (k-cores) structure.

In [45]:
```
# Compute the coreness of each vertex (i.e., k-core level)
coreness = g.coreness()

# Count how many nodes are in each k-core
core_counts = Counter(coreness)

# Sort by k
ks = sorted(core_counts)
counts = [core_counts[k] for k in ks]

# Plot the histogram
plt.figure(figsize=(8, 5))
plt.bar(ks, counts, color="steelblue", edgecolor="black")
plt.xlabel("k-core")
plt.ylabel("Number of vertices")
plt.title("Histogram of k-cores in the network")
plt.xticks(ks)
plt.grid(axis='y', linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()
```



In [46]:
```
plot_mds_graph(g, mds_coords, coreness, color_title="k-core", subtitle="k-core Colo
```



5.1.3. Density

The density of a network is defined as:

$$Density = \frac{2 \times edges}{n(n-1)}$$

That means, the proportion of actual edges in the network, in comparison to the maximum possible edges. In this case the density is equal to 0.62, which means that the network is very dense.

On the other hand, while computing the ego density for the most central nodes (betweenness centrality) we find that even though the node 54 is not the most central, it has the highest ego density. This means that even though it is not the most important node, it is very well connected to its neighbors.

In [47]:
```
# Compute density
density = g.density(loops=False)
print(f"Density of the graph: {density:.4f}")

# Ego density for most central vertex
top_indices = np.argsort(betweenness_w)[::-1][:5]
#print(top_indices)
print("\n 🔹 Top vertices Ego density:")
for idx in top_indices:
        v = g.vs[idx]
        ego_subgraph = g.induced_subgraph(g.neighborhood(v, order=1))
        ego_density = ego_subgraph.density(loops=False)
        print(f"Vertex {v['name']}: {ego_density:.4f}")
```

```
Density of the graph: 0.6171

 🔹 Top vertices Ego density:
Vertex 16: 0.6676
Vertex 13: 0.6475
Vertex 48: 0.6286
Vertex 59: 0.6471
Vertex 54: 0.7162
```

5.1.4. Cluster Coefficient (transitivity)

More than 60% of the potential triangles are actually present in the network. That means:

- The network is very dense locally
- Has a strong community structure
- Dominance relationships are relatively transitive (if A dominates B and B dominates C, A likely dominates C)

In [48]:
```
global_clustering = g.transitivity_undirected()
print(f"Global clustering coefficient (transitivity): {global_clustering:.4f}")
```

```
Global clustering coefficient (transitivity): 0.6599
```

5.2. Connectivity

In this section we are analysing whether the graph separates into distinct subgraphs. We already saw that the clustering coefficient is quite large, now we want to know if an arbitrary subset of k vertices (or edges) is removed, the remmaining subgraph will remain connected.

In our case, the removal of any subset of vertices/edges of cardinality lower than 17, leaves a subgraph that is connected. In other words, 17 vertices must be removed to disconnect the graph. This implies the graph is highly connected, as many vertices need to be removed to

break it apart. Also, it means that there are no small "bottlenecks". In adittion, this also means that there are not articulation points.

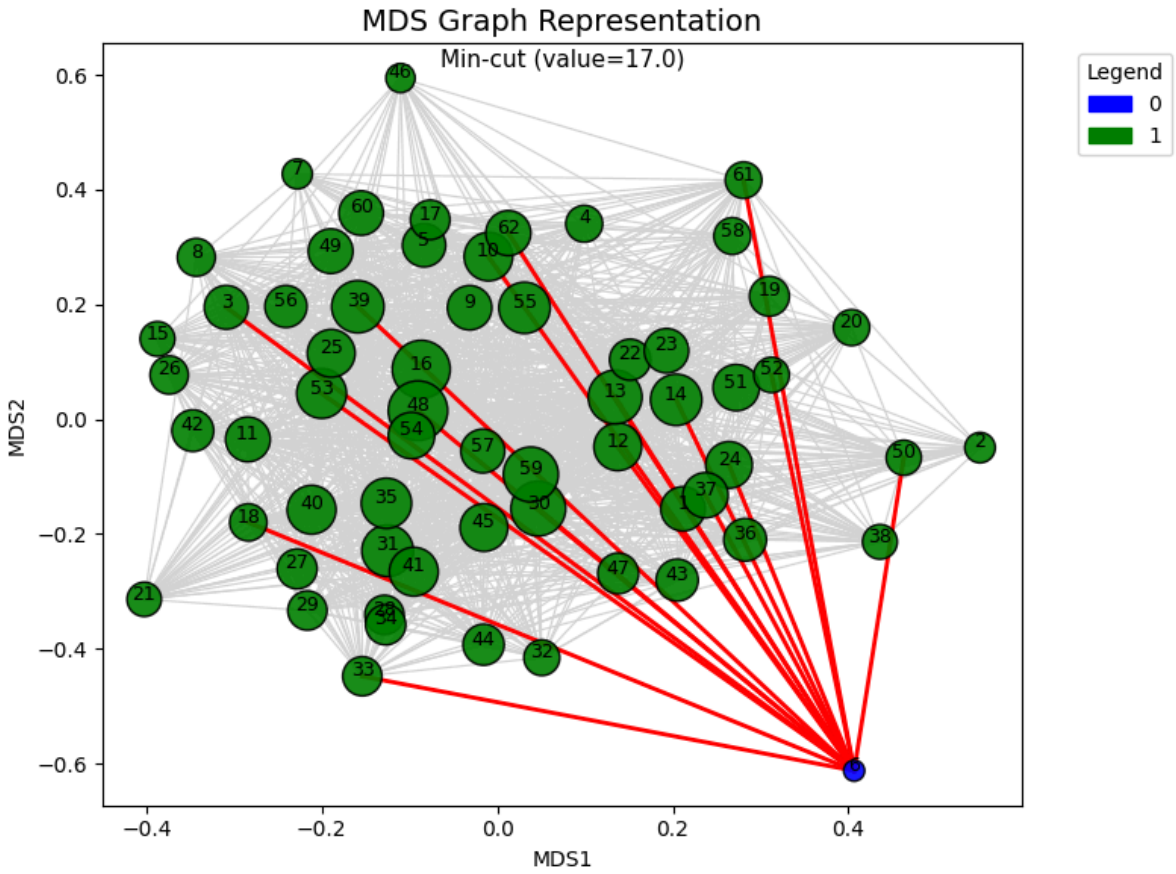
This result goes in concordance with the k-core analysis done before, when we detect that the smalles k-core was k=17 and the only member was node 6.

In [49]:
```
# Vertex Connectivity
vertex_conn = g.vertex_connectivity()
print(f"Vertex connectivity: {vertex_conn}")

articulation_points = g.articulation_points()
print(f"Articulation points (Names): {[g.vs[i]['name'] for i in articulation_points

cut = g.mincut()
print(f"Edge connectivity (mincut value): {cut.value}")
print(f"Edges in the cut: {cut.cut}")
print(f"Partition of vertices: {cut.partition}")
```

```
Vertex connectivity: 17
Articulation points (Names): []
Edge connectivity (mincut value): 17.0
Edges in the cut: [62, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176,
177, 178, 179, 180]
Partition of vertices: [[45], [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 1
5, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
36, 37, 38, 39, 40, 41, 42, 43, 44, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 5
7, 58, 59, 60, 61]]
```

In [50]:
```
# Initialize a list of partition labels (default: -1)
partition_labels = [-1] * len(g.vs)

# Assign partition numbers (0 and 1 for a 2-partition min-cut)
for partition_num, vertices in enumerate(cut.partition):
    for v in vertices:
        partition_labels[v] = partition_num  # Assign partition number to each vert
```

In [51]:
```
# Convert edge indices to (source, target) tuples
highlight_edges = [g.es[edge].tuple for edge in cut.cut]

# Now plot with corrected edges
plot_mds_graph(
    g, mds_coords, partition_labels,
    highlight_edges=highlight_edges,
    highlight_color='red',
    highlight_width=2.0,
    subtitle=f"Min-cut (value={cut.value})"
)
```



5.3. Community Detection

The principal aim of community detection in network analysis is to find disjoint subsets of vertices that demostrates a cohesiveness with respect to the underlying relational partners.

The Vertex Partition (of the vertex set V of the graph) ensures that in each subset the vertex are well connected among themselves anf that are relatively well separated from the vertices in other subsets.

An important measure introduced in this topic is the Modularity, where we compare the actual edges with the probability of the existance of an edge between two nodes. This last one is given by:

$$P_{uv} = \frac{d_u \times d_v}{2L}$$

where d_u and d_v are the degrees of the vertex u and v respectively.

We won't use the optimal modularity, as it is not feasible for networks that aren't particulary small. Is an NP-hard problem. To solve it, we should define an integer programming problem, and a solver able to deal with non linear terms. We will use heuristics methods instead, that balance the computational time and the quality of the solution.

In [52]:
```
def detect_communities(g, method="optimal_modularity", weights=None):
    """
    Detect communities in the graph using the specified algorithm.
    """
    if method == "fastgreedy":
        return g.community_fastgreedy(weights=weights).as_clustering()
    elif method == "Louvain":
```

```
        return g.community_multilevel(weights=weights)
    elif method == "label_propagation":
        return g.community_label_propagation(weights=weights)
    elif method == "Edge_Betweenness":
        return g.community_edge_betweenness(weights=weights).as_clustering()
    elif method == "walktrap":
        return g.community_walktrap(weights=weights).as_clustering()
    else:
        raise ValueError("Unknown community detection method.")

def print_communities(result, label="Community"):

    """
    Prints the number of communities and their members.
    """
    if hasattr(result, "as_clustering"):
        result = result.as_clustering()

    print(f"Number of communities ({label}): {len(result)}")
    for i, community in enumerate(result):
        print(f"Community {i + 1}: {community}")
```

5.3.1. Fast Greedy

Is a modification of the Optimal Modularity algorithm, but in this case maximize an approximation of the modularity in order to make it possible to apply it to large networks.

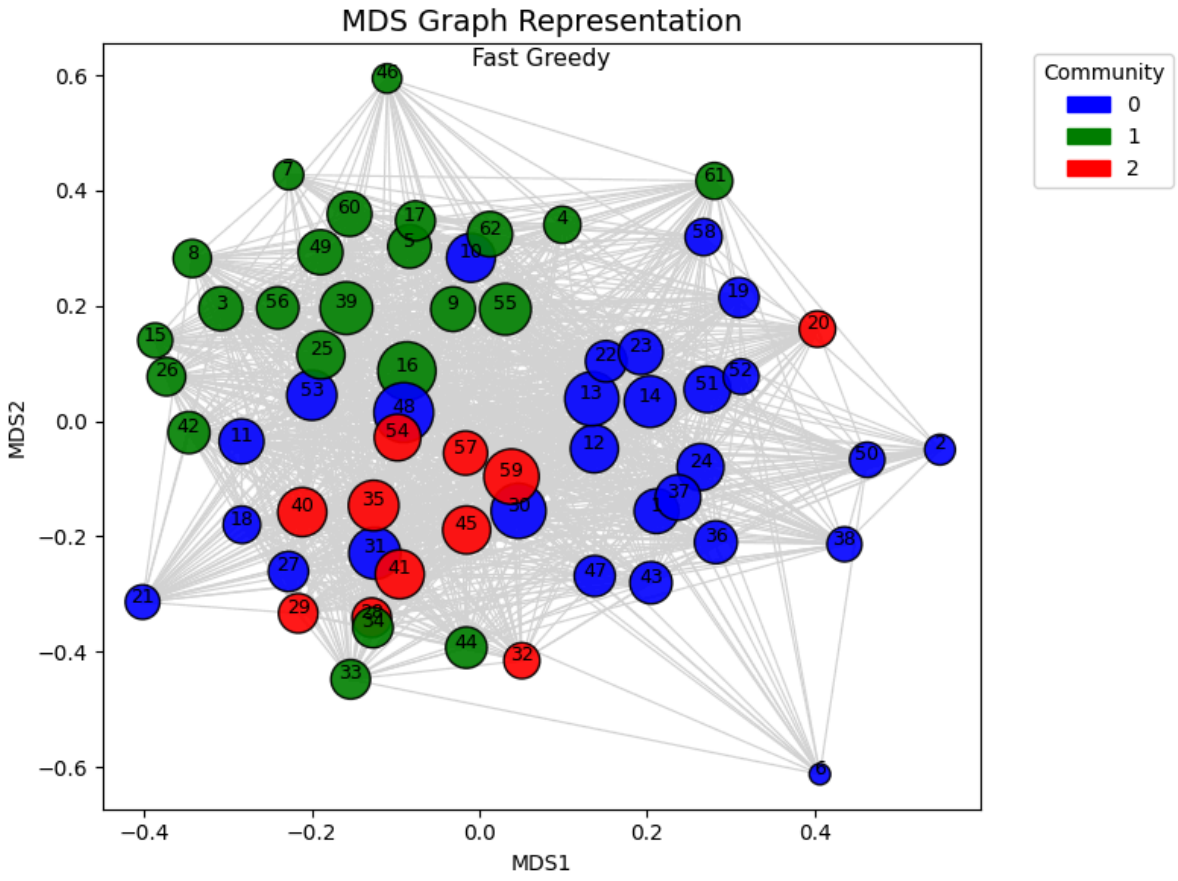
The results obtained in the unweighted and weighted case are similar. Both algorithms detect 3 communities, but the sizes of the communities are different. In the weighted case, the communities are more balanced.

- Unweighted: Sizes of 28, 23 and 11 vertices.
- Weighted: Sizes of 19, 22 and 21 vertices.

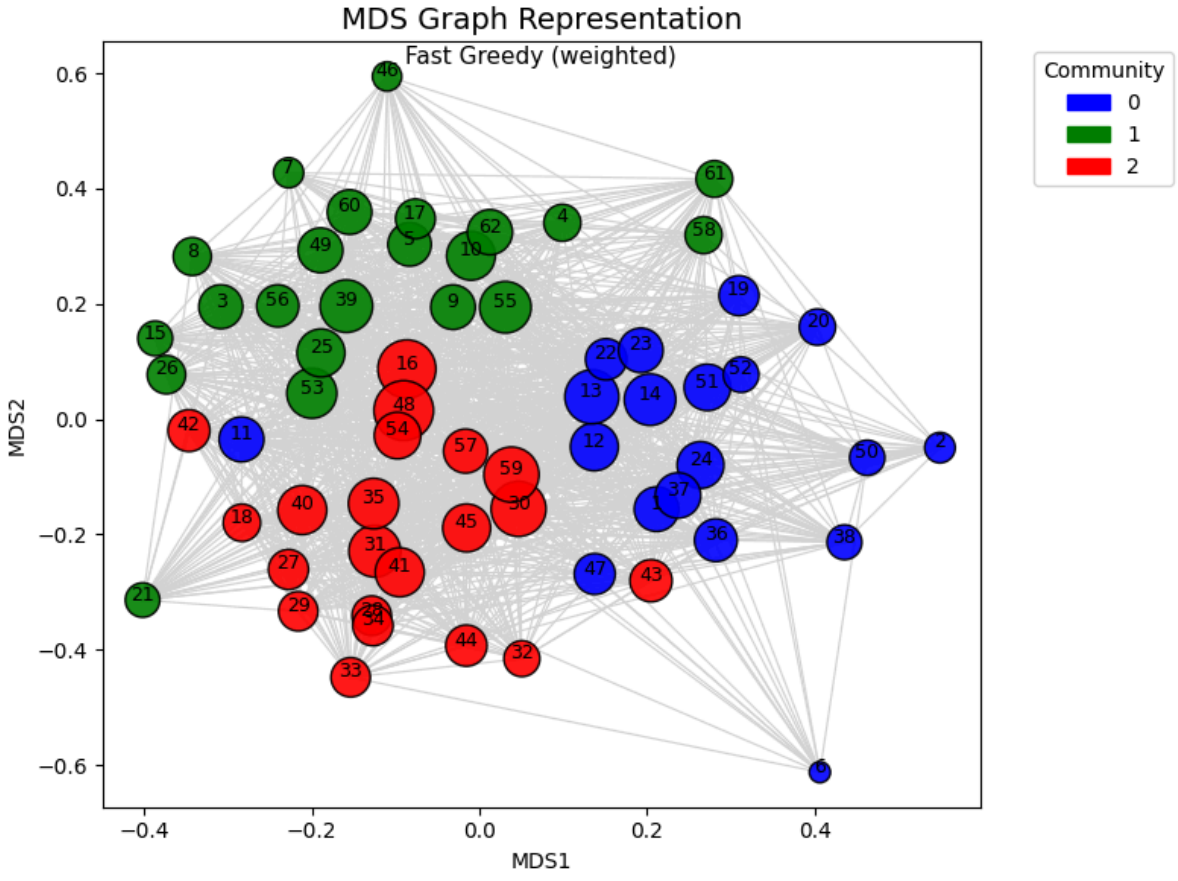
In [53]:
```
communities_fg = detect_communities(g, method="fastgreedy")
print_communities(communities_fg, "Fast Greedy")

communities_fg_w = detect_communities(g, method="fastgreedy", weights=g.es["weight"
print_communities(communities_fg_w, "Fast Greedy (weighted)")
```

```
Number of communities (Fast Greedy): 3
Community 1: [0, 1, 4, 5, 6, 7, 8, 10, 12, 13, 14, 15, 17, 18, 21, 22, 23, 26, 28,
29, 31, 32, 33, 37, 42, 45, 47, 49]
Community 2: [2, 3, 9, 16, 19, 25, 30, 34, 35, 41, 43, 44, 46, 48, 52, 53, 54, 56,
57, 58, 59, 60, 61]
Community 3: [11, 20, 24, 27, 36, 38, 39, 40, 50, 51, 55]
Number of communities (Fast Greedy (weighted)): 3
Community 1: [0, 1, 5, 6, 7, 8, 11, 13, 14, 15, 21, 22, 23, 28, 31, 32, 33, 45, 4
7]
Community 2: [2, 3, 4, 12, 16, 30, 34, 35, 37, 42, 43, 44, 46, 48, 53, 54, 56, 57,
58, 59, 60, 61]
Community 3: [9, 10, 17, 18, 19, 20, 24, 25, 26, 27, 29, 36, 38, 39, 40, 41, 49, 5
0, 51, 52, 55]
```

In [54]:
```
#fastgreedy_membership = communities_fg.as_clustering()
plot_mds_graph(g, mds_coords, communities_fg.membership, color_title="Community", s
```



In [55]:
```
#fastgreedy_membership_w = communities_fg_w.as_clustering()
plot_mds_graph(g, mds_coords, communities_fg_w.membership, color_title="Community",
```



5.3.2. Louvain

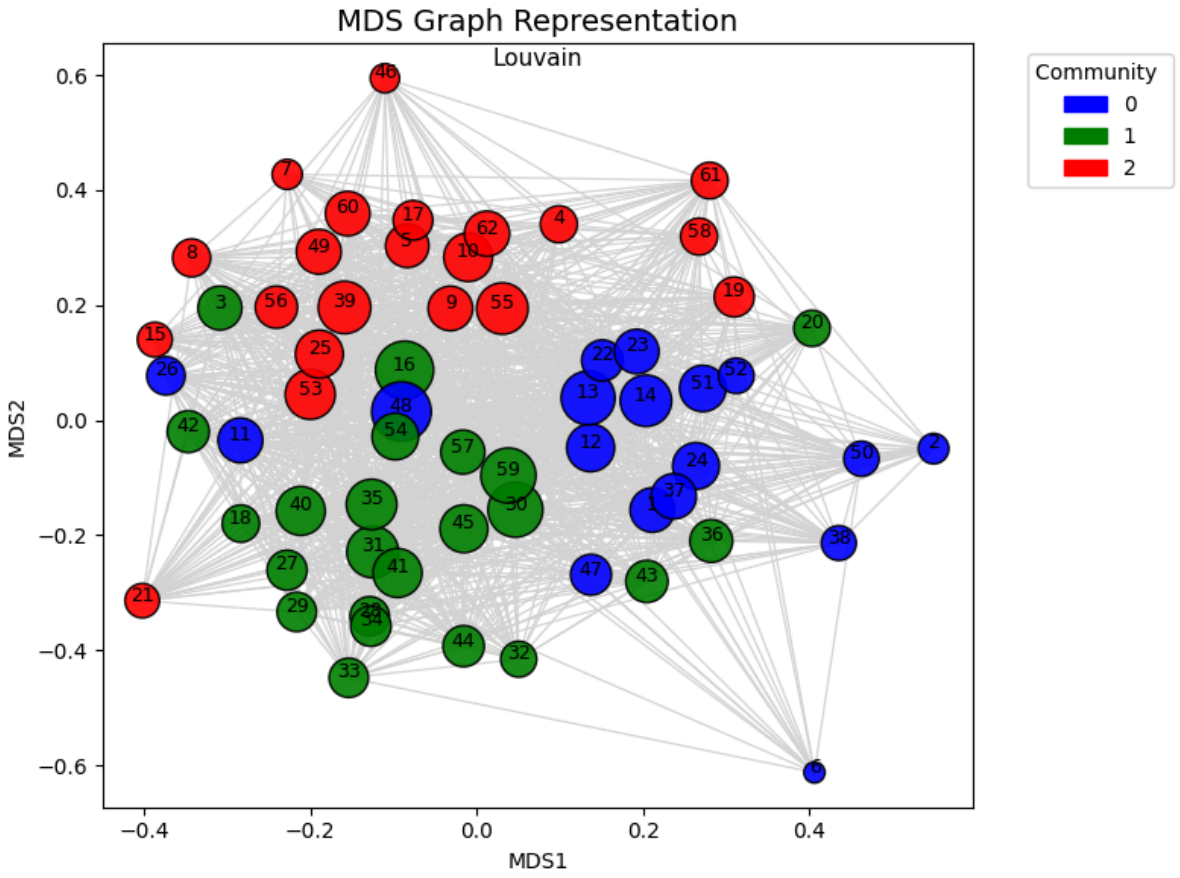
Louvain algorithm runs a hierarchical approach (from local to global communities) until an optimal modularity value is attend.

In both cases (weighted and unweighted) Louvain algorithm also divides the network in 3 communities. Also the changes between communities is really small and is similar to the previously obtained results.

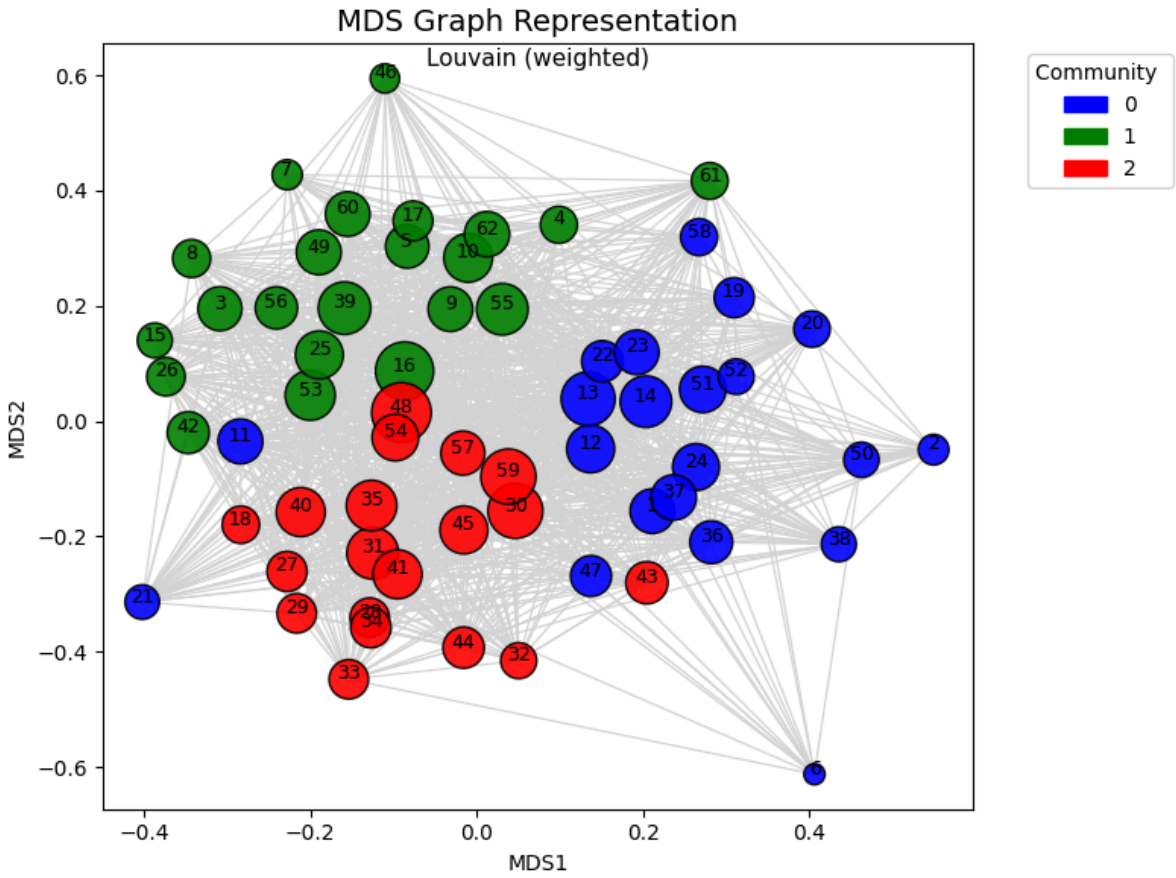
In [56]:
```
communities_louvain = detect_communities(g, method="Louvain")
print_communities(communities_louvain, "Louvain")

communities_louvain_w = detect_communities(g, method="Louvain", weights=g.es["weigh
print_communities(communities_louvain_w, "Louvain (weighted)")
```

```
Number of communities (Louvain): 3
Community 1: [0, 1, 5, 6, 7, 8, 13, 14, 15, 16, 22, 23, 28, 29, 31, 32, 33, 45]
Community 2: [2, 9, 10, 11, 17, 18, 19, 20, 21, 24, 25, 26, 27, 36, 38, 39, 40, 4
1, 49, 50, 51, 52, 55]
Community 3: [3, 4, 12, 30, 34, 35, 37, 42, 43, 44, 46, 47, 48, 53, 54, 56, 57, 5
8, 59, 60, 61]
Number of communities (Louvain (weighted)): 3
Community 1: [0, 1, 5, 6, 7, 8, 11, 12, 13, 14, 15, 21, 22, 23, 28, 31, 32, 33, 3
7, 45, 47]
Community 2: [2, 3, 4, 9, 16, 25, 30, 34, 35, 42, 43, 44, 46, 48, 53, 54, 56, 57,
58, 59, 60, 61]
Community 3: [10, 17, 18, 19, 20, 24, 26, 27, 29, 36, 38, 39, 40, 41, 49, 50, 51,
52, 55]
```

In [57]:
```
plot_mds_graph(g, mds_coords, communities_louvain.membership, color_title="Communit
```



In [58]:
```
plot_mds_graph(g, mds_coords, communities_louvain_w.membership, color_title="Commur
```



5.3.3. Label Propagation

The Label Propagation Algorithm decide the community of each vertex taking into account the communities of its neighbors. Unfortunately in our case, we did not obtain an informative result because it masked all the vertex in only one community (taking into account or not the weights), as it tend to happen in dense networks.

In [59]:
```
communities_lp = detect_communities(g, method="label_propagation")
print_communities(communities_lp, "Label Propagation Algorithm")

communities_lp_w = detect_communities(g, method="label_propagation", weights=g.es['
print_communities(communities_lp_w, "Label Propagation Algorithm (weighted)")
```

```
Number of communities (Label Propagation Algorithm): 1
Community 1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1
9, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39,
40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 6
0, 61]
Number of communities (Label Propagation Algorithm (weighted)): 1
Community 1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1
9, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39,
40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 6
0, 61]
```

5.3.4. Edge Betweeness

The Edge betweeness algorithm creates communities using a divisive method based on deleting central edges.

In this case the result is also not desirable because in the unweighted case it creates 28 communities, 1 with most of the central nodes, and the rest of the communities contains a

single vertex. In the weighted case it masked all the vertices in a single community.

In [60]:
```
communities_eb = detect_communities(g, method="Edge_Betweenness")
print_communities(communities_eb, "Edge Betweenness Algorithm")

communities_eb_w = detect_communities(g, method="Edge_Betweenness", weights=g.es["w
print_communities(communities_eb_w, "Edge Betweenness Algorithm (weighted)")

#eb_membership = dendogram_eb.as_clustering()
#eb_membership_w = dendogram_eb_w.as_clustering()
```

```
Number of communities (Edge Betweenness Algorithm): 28
Community 1: [0, 2, 5, 6, 7, 8, 9, 11, 13, 15, 16, 17, 18, 19, 20, 21, 24, 25, 26,
27, 29, 30, 35, 36, 38, 39, 40, 41, 42, 48, 49, 52, 53, 55, 57]
Community 2: [1]
Community 3: [3]
Community 4: [4]
Community 5: [10]
Community 6: [12]
Community 7: [14]
Community 8: [22]
Community 9: [23]
Community 10: [28]
Community 11: [31]
Community 12: [32]
Community 13: [33]
Community 14: [34]
Community 15: [37]
Community 16: [43]
Community 17: [44]
Community 18: [45]
Community 19: [46]
Community 20: [47]
Community 21: [50]
Community 22: [51]
Community 23: [54]
Community 24: [56]
Community 25: [58]
Community 26: [59]
Community 27: [60]
Community 28: [61]
Number of communities (Edge Betweenness Algorithm (weighted)): 1
Community 1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 1
9, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39,
40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 6
0, 61]
```

5.3.5. Walktrap

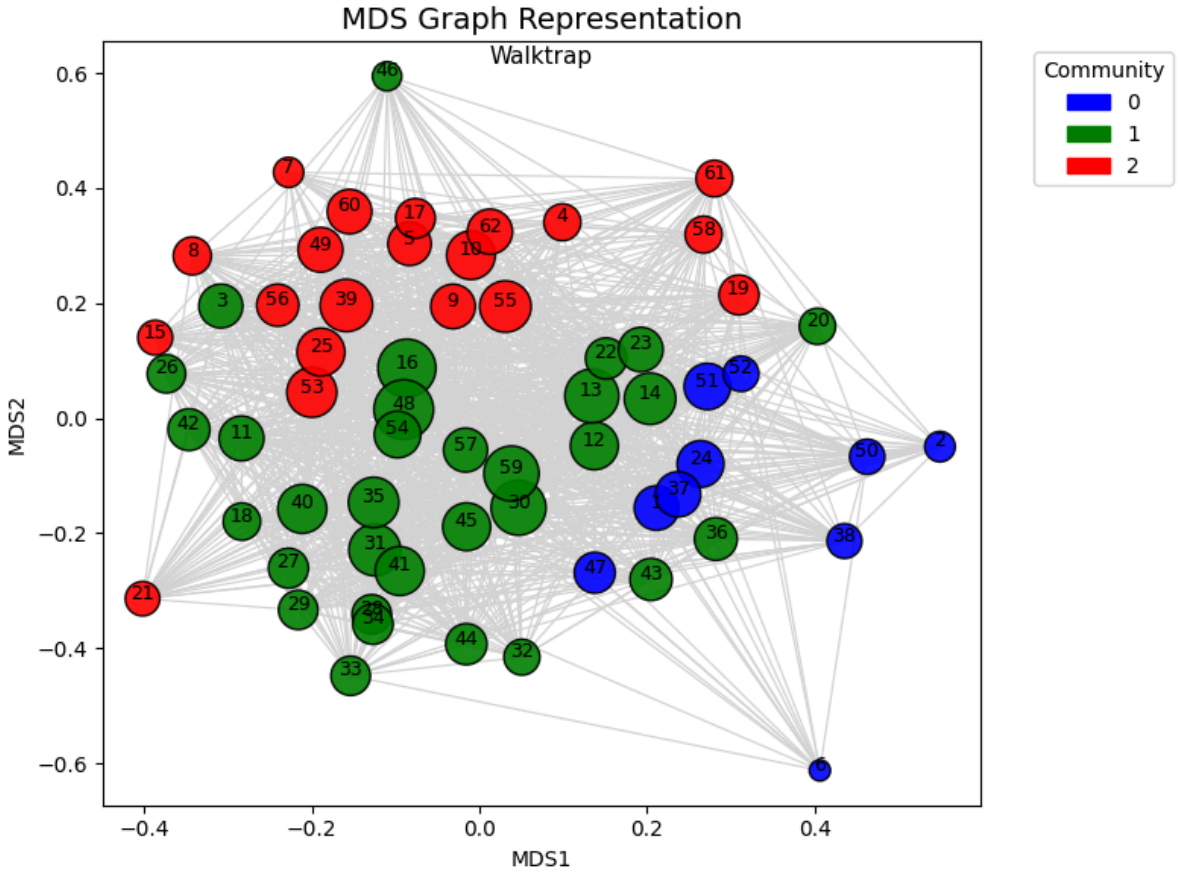
Walktrap algorithm uses hierarchical clustering with a certain distance between the vertices. This distance is defined as 1 minus the probability of "a random walk of a certain (small) length starting in the first vertex ends in the second".

In this case, we also obtain 3 communities, in the unweighted case we obtain a bigger community and in the weighted case we obtain closer sizes.

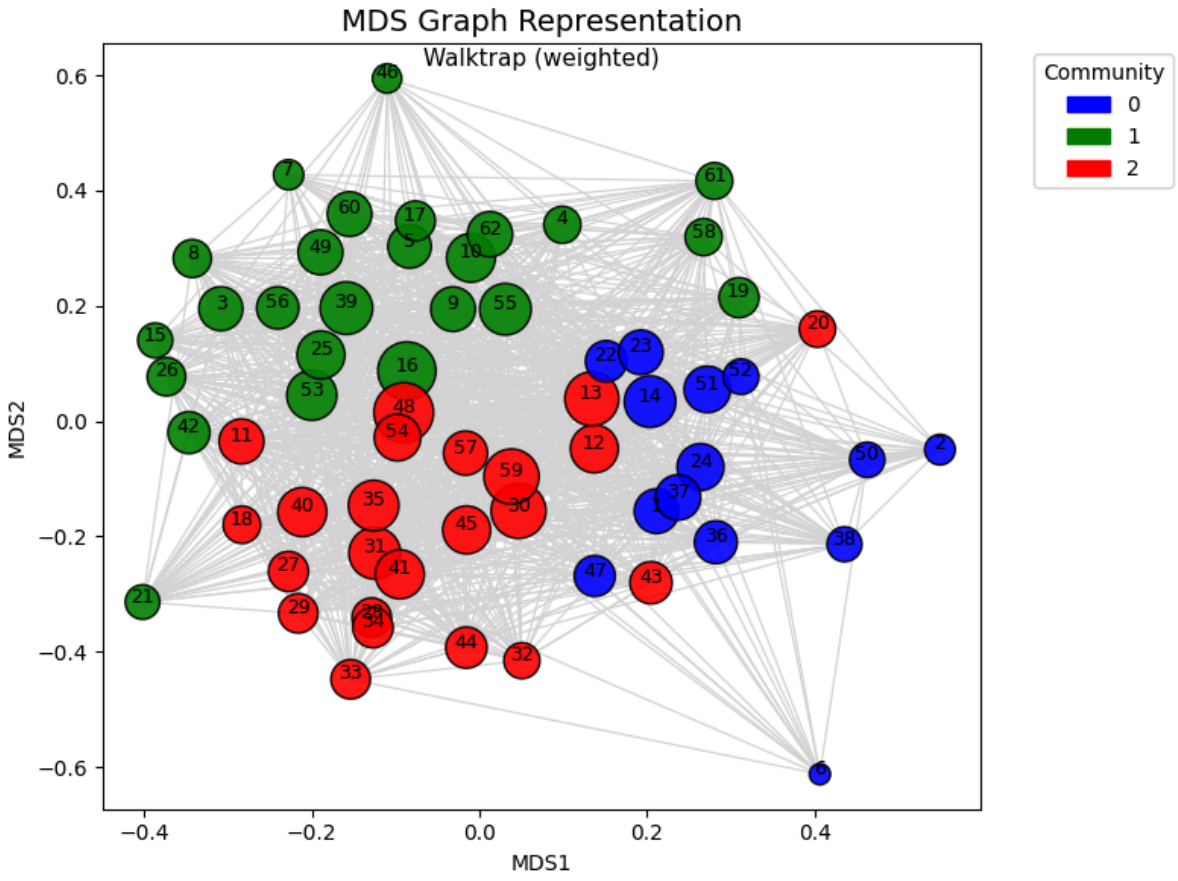
In [61]:
```
communities_walkt = detect_communities(g, method="walktrap")
print_communities(communities_walkt, "Walktrap Algorithm")

communities_walkt_w = detect_communities(g, method="walktrap", weights=g.es["weight
print_communities(communities_walkt_w, "Walktrap Algorithm (weighted)")
```

```
Number of communities (Walktrap Algorithm): 3
Community 1: [0, 1, 15, 22, 23, 28, 31, 32, 33, 45]
Community 2: [2, 5, 6, 7, 8, 9, 10, 11, 13, 14, 16, 17, 18, 19, 20, 21, 24, 25, 2
6, 27, 29, 36, 38, 39, 40, 41, 49, 50, 51, 52, 54, 55]
Community 3: [3, 4, 12, 30, 34, 35, 37, 42, 43, 44, 46, 47, 48, 53, 56, 57, 58, 5
9, 60, 61]
Number of communities (Walktrap Algorithm (weighted)): 3
Community 1: [0, 1, 8, 13, 14, 15, 21, 22, 23, 28, 31, 32, 33, 45]
Community 2: [2, 3, 4, 9, 12, 16, 25, 30, 34, 35, 37, 42, 43, 44, 46, 47, 48, 53,
54, 56, 57, 58, 59, 60, 61]
Community 3: [5, 6, 7, 10, 11, 17, 18, 19, 20, 24, 26, 27, 29, 36, 38, 39, 40, 41,
49, 50, 51, 52, 55]
```

In [62]:
```
#walktrap_membership = dendogram_walkt.as_clustering()
plot_mds_graph(g, mds_coords, communities_walkt.membership, color_title="Community'
```



In [63]:
```
#walktrap_membership_w = dendogram_walkt_w.as_clustering()
plot_mds_graph(g, mds_coords, communities_walkt_w.membership, color_title="Communit
```



For the algorithms that we obtain a good result (3 communities), the modularity is really close. However, in the unweighted and weighted case, the best options seems to be Louvain algorithms.

In [64]:
```
algorithms = ["Fast Greedy", "Louvain", "Label Propagation", "Edge Betweeness", "Wa

modularity = [communities_fg.modularity, communities_louvain.modularity,
              communities_lp.modularity, communities_eb.modularity, communities_wal


modularity_weighted = [communities_fg_w.modularity, communities_louvain_w.modularit
              communities_lp_w.modularity, communities_eb_w.modularity, communities

df = pd.DataFrame({
    "Algorithm": algorithms,
    "Modularity": modularity,
    "Weighted Modularity": modularity_weighted
})

# Display the table
print(df.to_string(index=False))
```

```
        Algorithm  Modularity  Weighted Modularity
      Fast Greedy    0.071611             0.182846
          Louvain    0.096506             0.187193
Label Propagation    0.000000             0.000000
  Edge Betweeness    0.011177             0.000000
         Walktrap    0.084542             0.174141
```

5.4. Assortativity Coefficient

The assortativity coefficient is a measure of the tendency of nodes to connect to other nodes of similar degree. The assortativity coefficient is a value between -1 and 1, where -1 means

that the nodes tend to connect to nodes with different degrees, 0 means that there is no correlation between the degrees of the nodes and 1 means that the nodes tend to connect to nodes with similar degrees.

In our case, the assotativity coefficient is -0.0726, which indicates a neutral network with slight tendency toward disassortativity. In this case, Macaques with many dominance relationships have a slight tendency to connect with monkeys that have fewer dominance relationships, and vice versa. This pattern is consistent with hierarchical social structures where high-ranking individuals (with many connections) interact with lower-ranking individuals (with fewer connections).

In [65]:
```
# Degree assortativity
r = g.assortativity_degree(directed=False)
print(f"Degree assortativity coefficient: {r:.4f}")
```

```
Degree assortativity coefficient: -0.0726
```

6. Random graph generation experiments

In [66]:
```
def generate_erdos_renyi_graph(n, m):
    """Generate an Erdos-Renyi graph with n nodes and m edges."""
    g = ig.Graph()
    g.add_vertices(n)
    possible_edges = [(i, j) for i in range(n) for j in range(i+1, n)]
    chosen_edges = random.sample(possible_edges, m)
    g.add_edges(chosen_edges)
    return g

def generate_grg_with_fixed_degree_sequence(degree_sequence):
    """Generate a random graph with a fixed degree sequence using configuration mod
    g = ig.Graph.Degree_Sequence(degree_sequence, method="vl")
    return g

def experiment_random_graphs_communities(base_graph, methods, n_samples=1000, seed=
    os.makedirs("./data", exist_ok=True)

    file_er = os.path.join("data", experiment_names[0] + "_" + str(n_samples) + ".p
    file_grg = os.path.join("data", experiment_names[1] + "_" + str(n_samples) + ".

    results_er = None
    results_grg = None

    if os.path.exists(file_er):
        with open(file_er, "rb") as f:
            results_er = pickle.load(f)
        print(f"Loaded community_experiment Erdős-Rényi results from {file_er}")
    else:
        results_er = defaultdict(list)

    if os.path.exists(file_grg):
        with open(file_grg, "rb") as f:
            results_grg = pickle.load(f)
        print(f"Loaded community_experiment GRG results from {file_grg}")
    else:
        results_grg = defaultdict(list)

    if results_er and results_grg:
        return results_er, results_grg
```

```
    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    n = base_graph.vcount()
    m = base_graph.ecount()
    degree_sequence = base_graph.degree()

    for i in range(n_samples):
        if (i + 1) % 10 == 0 or i == 0 and verbose:
            print(f"Sample {i + 1}/{n_samples}")
        g_er = generate_erdos_renyi_graph(n, m)
        g_grg = generate_grg_with_fixed_degree_sequence(degree_sequence)

        for method in methods:
            try:
                com_er = detect_communities(g_er, method=method)
                results_er[method].append(len(com_er))
            except Exception:
                results_er[method].append(None)

            try:
                com_grg = detect_communities(g_grg, method=method)
                results_grg[method].append(len(com_grg))
            except Exception:
                results_grg[method].append(None)

    # Save results
    with open(file_er, "wb") as f:
        pickle.dump(results_er, f)
    with open(file_grg, "wb") as f:
        pickle.dump(results_grg, f)

    print(f"Saved ER results to {file_er}")
    print(f"Saved GRG results to {file_grg}")

    return results_er, results_grg

def experiment_random_graphs_structure_metrics(base_graph, n_samples=1000, seed=Nor
                                               experiment_names=["structure_metrics
    import os
    import pickle
    from collections import defaultdict

    os.makedirs("./data", exist_ok=True)

    file_er = os.path.join("data", experiment_names[0] + f"_{n_samples}.pkl")
    file_grg = os.path.join("data", experiment_names[1] + f"_{n_samples}.pkl")

    results_er = []
    results_grg = []

    # Load if available
    if os.path.exists(file_er):
        with open(file_er, "rb") as f:
            results_er = pickle.load(f)
        print(f"Loaded structure metrics ER results from {file_er}")
    if os.path.exists(file_grg):
        with open(file_grg, "rb") as f:
            results_grg = pickle.load(f)
        print(f"Loaded structure metrics GRG results from {file_grg}")

    # If both exist, return
    if results_er and results_grg:
```

```
        return results_er, results_grg

    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    n = base_graph.vcount()
    m = base_graph.ecount()
    degree_sequence = base_graph.degree()

    for i in range(n_samples):
        if verbose and ((i + 1) % 10 == 0 or i == 0):
            print(f"Sample {i + 1}/{n_samples}")

        g_er = generate_erdos_renyi_graph(n, m)
        g_grg = generate_grg_with_fixed_degree_sequence(degree_sequence)

        for graph, results in zip([g_er, g_grg], [results_er, results_grg]):
            try:
                if not graph.is_connected():
                    gc = graph.clusters().giant()
                else:
                    gc = graph
                metrics = {
                    "avg_path_length": gc.average_path_length(),
                    "diameter": gc.diameter(),
                    "density": graph.density(),
                    "clustering": graph.transitivity_undirected(),
                    "n_components": len(graph.clusters()),
                    "largest_component_size": gc.vcount(),
                    "assortativity": graph.assortativity_degree(directed=False),
                    "vertex_connectivity": graph.vertex_connectivity(),
                    "edge_mincut": graph.mincut().value,
                }
            except Exception:
                metrics = {
                    "avg_path_length": None,
                    "diameter": None,
                    "density": None,
                    "clustering": None,
                    "n_components": None,
                    "largest_component_size": None,
                    "assortativity": None,
                    "vertex_connectivity": None,
                    "edge_mincut": None,
                }

            results.append(metrics)

    # Save results
    with open(file_er, "wb") as f:
        pickle.dump(results_er, f)
    with open(file_grg, "wb") as f:
        pickle.dump(results_grg, f)

    print(f"Saved structure metrics ER results to {file_er}")
    print(f"Saved structure metrics GRG results to {file_grg}")

    return results_er, results_grg

def plot_all_relative_frequencies_grid(results_er, results_grg):
    methods = list(results_er.keys())
    num_methods = len(methods)
```

```
    ncols = 2
    nrows = (num_methods + 1) // 2

    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(10, 4 * nrows), cor
    axes = axes.flatten()

    for idx, method in enumerate(methods):
        ax = axes[idx]

        # Extract counts
        counts_er = [v for v in results_er[method] if v is not None]
        counts_grg = [v for v in results_grg[method] if v is not None]

        # Determine bin range
        all_counts = counts_er + counts_grg
        bins = sorted(set(all_counts))
        hist_er, _ = np.histogram(counts_er, bins=bins + [bins[-1] + 1], density=Tr
        hist_grg, _ = np.histogram(counts_grg, bins=bins + [bins[-1] + 1], density=

        x = np.arange(len(bins))
        width = 0.4

        ax.bar(x - width / 2, hist_er, width=width, color="deepskyblue", edgecolor=
        ax.bar(x + width / 2, hist_grg, width=width, color="mediumorchid", edgecolo

        #ax.set_xticks(x)
        #ax.set_xticklabels(bins)
        if len(bins) > 20:
            tick_indices = np.arange(0, len(bins), max(1, len(bins) // 15))
            ax.set_xticks(tick_indices)
            ax.set_xticklabels([bins[i] for i in tick_indices], rotation=45, ha='ri
        else:
            ax.set_xticks(x)
            ax.set_xticklabels(bins)

        ax.set_title(method.replace("_", " ").title())
        ax.set_xlabel("Number of communities")
        ax.set_ylabel("Relative frequencies")

        if idx == 0:
            ax.legend()

    # Hide any extra subplot
    for j in range(len(methods), len(axes)):
        fig.delaxes(axes[j])

    plt.show()
```

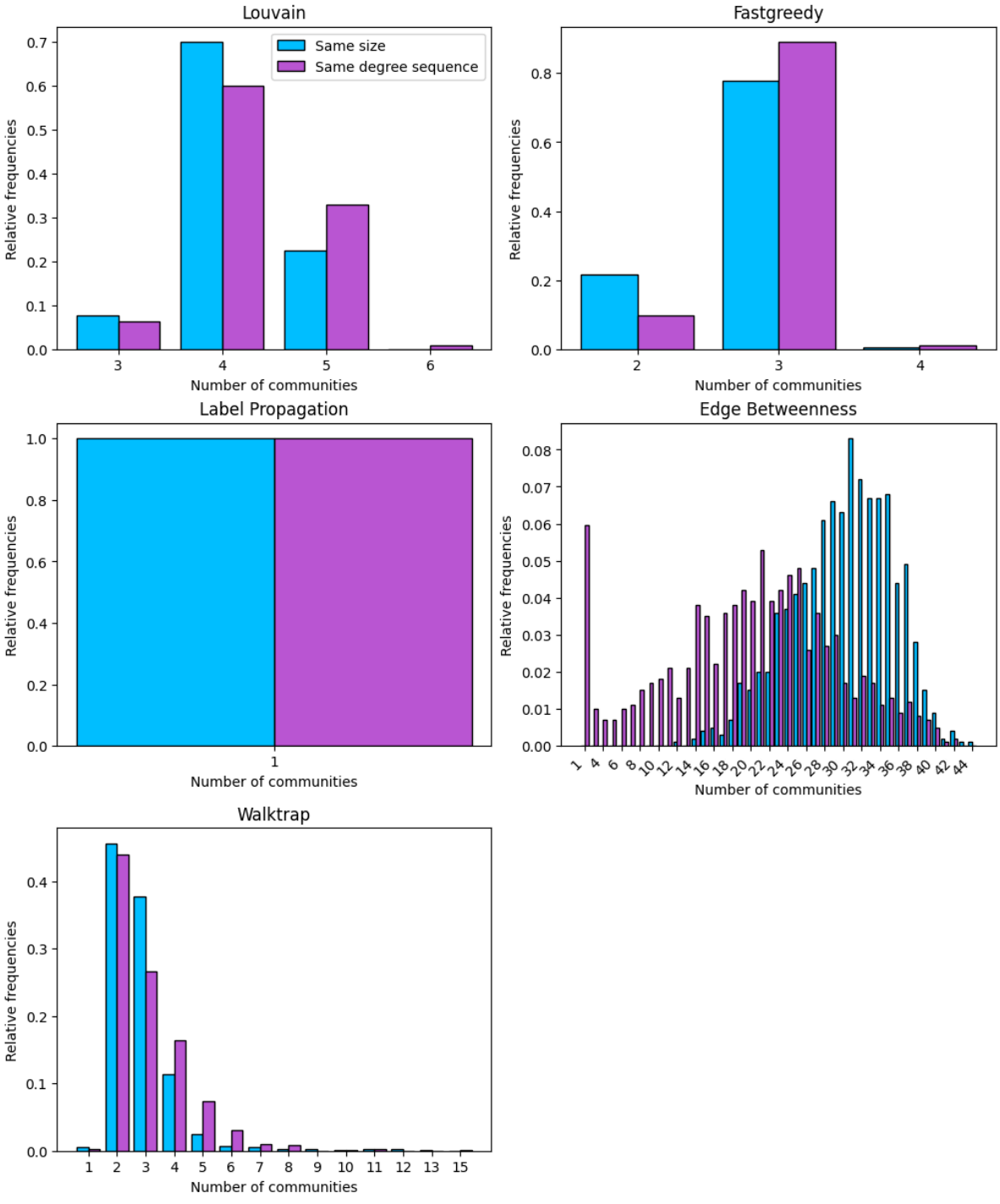
6.1. Community assessment

Now we will perform some experiments to see how common the structure of our network is and the clustering results obtained. We will generate 1000 random graphs with the same number of vertices and edges as our network, and 1000 with the same number of vertices and degree distribution. We will perform on them the same clustering algorithms and check the frequencies obtained. With the distribution of the results, we can tell how reliable the results obtained in our network are.

In [67]:
```
methods = ["Louvain", "fastgreedy", "label_propagation", "Edge_Betweenness", "walkt
results_er_comm, results_grg_comm = experiment_random_graphs_communities(g, methods
```

```
Loaded community_experiment Erdős–Rényi results from data\community_experiment_res
ults_ER_1000.pkl
Loaded community_experiment GRG results from data\community_experiment_results_GRG
_1000.pkl
```

In [68]:
```
plot_all_relative_frequencies_grid(results_er_comm, results_grg_comm)
```



Let's compare our network with a random graph. To be fair, as the Erdos-Renyi model generates random graphs without weights, we will use the unweighted version of our network. Let's recap the results obtained in our network:

Clustering Method	Number of Clusters
Louvain	3
Fast Greedy	3
Label Propagation	1

Clustering Method	Number of Clusters
Edge Betweenness	28
Walktrap	3

- Louvain obtained 3 communities, but in our experiment we can see how this is quite unlikely (less than 10% on both methods). So our result is not very reliable.
- Fast Greedy also obtained 3 communities, but in this case the result is the most common with a big difference in the experiment (around 80% in the same size experiment and more in the same degree distribution).
- Label Propagation obtained 1 community, which is also the only result obtained in the experiment. As is expected in dense networks.
- Edge Betweenness obtained 28 communities, which is not uncommon in the experiment even thought is not the mode in any of the methods, and there is more variability than in the other methods.
- Walktrap obtained 3 communities, which is also quite likely (almost 40% and almost 30% for each experiment, respectively).

6.2. Structural properties assessment

We also performed a similar experiment to validate other structural properties of the network. Again, we will generate 1000 random graphs with the same number of vertices and edges as our network, and 1000 with the same number of vertices and degree distribution. We will compute the following properties for each graph and use the mean to compare.

- Average path length.
- Diameter.
- Density.
- Clustering coefficient.
- Number of components.
- Largest component size.
- Assortativity.
- Vertex connectivity.
- Edge min cut.

In [69]:
```
results_er_str, results_grg_str = experiment_random_graphs_structure_metrics(g, n_s
```

```
Loaded structure metrics ER results from data\structure_metrics_ER_1000.pkl
Loaded structure metrics GRG results from data\structure_metrics_GRG_1000.pkl
```

In [70]:
```
macaque_metrics = {
    "avg_path_length": g.average_path_length(),
    "diameter": g.diameter(),
    "density": g.density(),
    "clustering": g.transitivity_undirected(),
    "n_components": len(g.clusters()),
    "largest_component_size": g.clusters().giant().vcount(),
    "assortativity": g.assortativity_degree(directed=False),
    "vertex_connectivity": g.vertex_connectivity(),
    "edge_mincut": g.mincut().value
}
macaque_metrics_df =pd.DataFrame.from_dict(macaque_metrics, orient='index', columns
```

```
C:\Users\danie\AppData\Local\Temp\ipykernel_19860\680991276.py:6: DeprecationWarni
ng: Graph.clusters() is deprecated; use Graph.connected_components() instead
  "n_components": len(g.clusters()),
C:\Users\danie\AppData\Local\Temp\ipykernel_19860\680991276.py:7: DeprecationWarni
ng: Graph.clusters() is deprecated; use Graph.connected_components() instead
  "largest_component_size": g.clusters().giant().vcount(),
```

In [71]:
```
df_er = pd.DataFrame(results_er_str)
df_grg = pd.DataFrame(results_grg_str)

means_er = df_er.mean(numeric_only=True)
means_grg = df_grg.mean(numeric_only=True)

summary_df = pd.DataFrame({
    "ER": means_er,
    "GRG": means_grg
})

summary_df = summary_df.join(macaque_metrics_df)

summary_df = summary_df.round(4)

display(summary_df)
```

	ER	GRG	Real
avg_path_length	1.3829	1.3829	1.3829
diameter	2.0000	2.0000	2.0000
density	0.6171	0.6171	0.6171
clustering	0.6168	0.6510	0.6599
n_components	1.0000	1.0000	1.0000
largest_component_size	62.0000	62.0000	62.0000
assortativity	-0.0321	-0.0833	-0.0726
vertex_connectivity	28.6980	17.0000	17.0000
edge_mincut	28.6980	17.0000	17.0000

As we can see, most of the results are the same or really close to the ones obtained in our network. The only difference is the vertex connectivity and edge min cut, which are higher in the random graph. This is expected, as the random graph generated with the same number of vertices and edges spreads the connections evenly. In our case and in the generalized random graph, there are individuals with a lot of connections and others with few connections. If we remember our min cut, we needed to remove 17 edges to disconnect the graph. Those edges belonged to an individual with relatively low connectivity. It is not surprising that the graphs generated with the same degree distribution end up having the same min cut. Due to the high density, the easiest way to disconnect the graph is by removing the edges of the individual with less degree.

6.3. Small world properties assessment

Next, we are going to check if our network has small-world properties. We will run an experiment in which we will generate 10000 random graphs using the Erdos-Renyi model

with the same number of vertices and edges as our network. We will compute the average path length and the clustering coefficient for each graph and use the mean to compare. Then we will plot the histogram of the results obtained and the results of our network. If the network has small world properties, we should see that the average path length is the same and the clustering coefficient is higher.

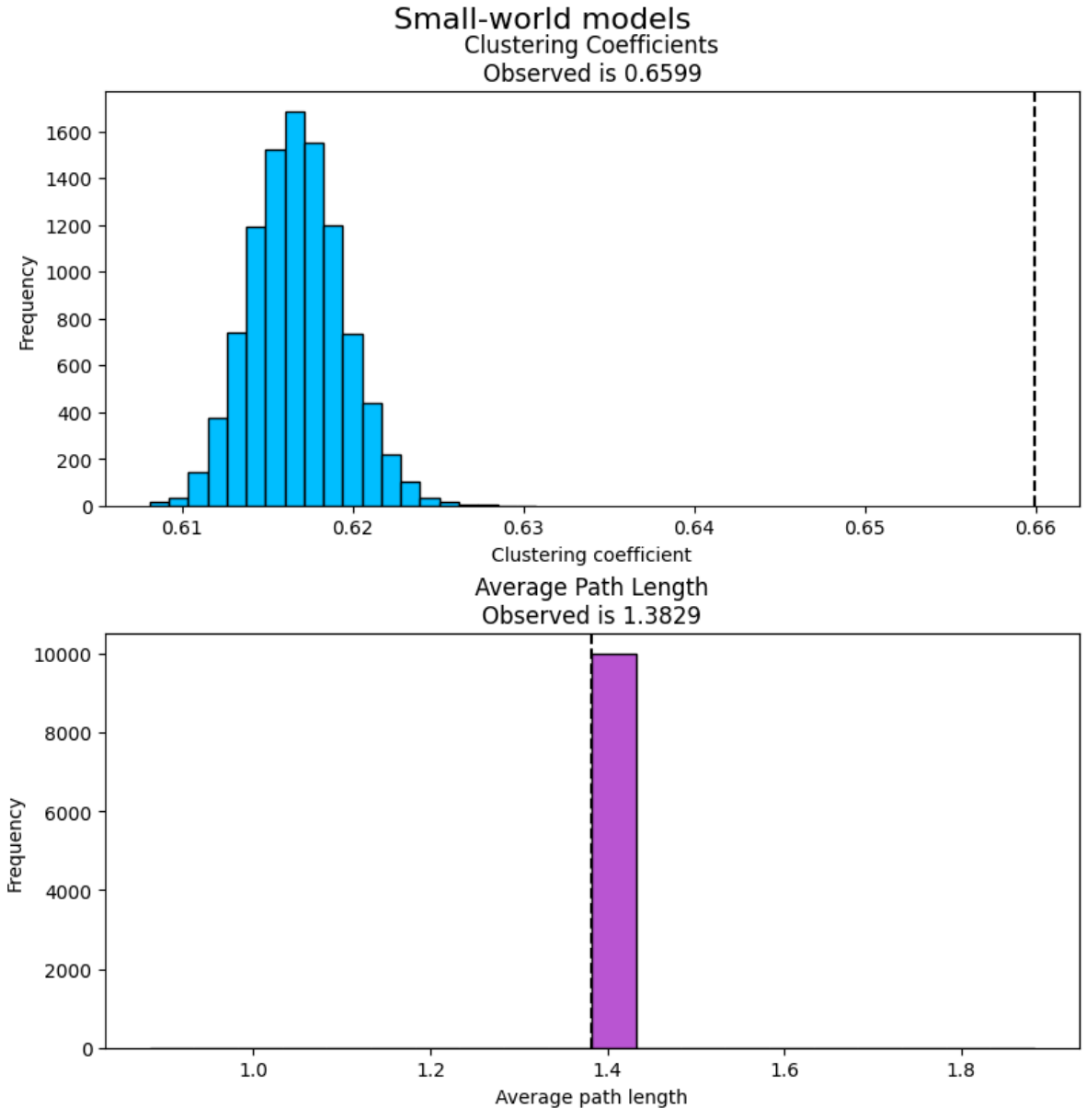
In [72]:
```
def generate_random_graph_stats(base_graph, n_samples=1000, seed=None, verbose=Fals
    n = base_graph.vcount()
    m = base_graph.ecount()

    os.makedirs("data", exist_ok=True)

    file_path = os.path.join("data", f"{experiment_name}_{n_samples}.pkl")

    # If file exists, load it
    if os.path.exists(file_path):
        with open(file_path, "rb") as f:
            print(f"Loaded from {file_path}")
            return pickle.load(f)

    if seed is not None:
        random.seed(seed)
        np.random.seed(seed)

    clustering_random = []
    path_length_random = []

    for i in range(n_samples):
        if (i + 1) % 10 == 0 or i == 0 and verbose:
            print(f"Sample {i + 1}/{n_samples}")

        g_rand = generate_erdos_renyi_graph(n, m)

        try:
            clustering = g_rand.transitivity_undirected()
            path_length = g_rand.average_path_length()
        except:
            clustering = None
            path_length = None

        clustering_random.append(clustering)
        path_length_random.append(path_length)

    result = {
        "clustering": clustering_random,
        "path_length": path_length_random
    }

    with open(file_path, "wb") as f:
        pickle.dump(result, f)
        print(f"Saved to {file_path}")

    return result

def plot_smallworld_histograms(results_smallworld, observed_clustering, observed_pa
    clustering = [v for v in results_smallworld["clustering"] if v is not None]
    path_length = [v for v in results_smallworld["path_length"] if v is not None]

    fig, axs = plt.subplots(2, 1, figsize=(8, 8), constrained_layout=True)

    # Clustering coefficient histogram
    axs[0].hist(clustering, bins=20, color='deepskyblue', edgecolor='black')
```

```
    axs[0].axvline(observed_clustering, color='black', linestyle='--')
    axs[0].set_title(f"Clustering Coefficients\nObserved is {observed_clustering:.4
    axs[0].set_xlabel("Clustering coefficient")
    axs[0].set_ylabel("Frequency")

    # Average path length histogram
    axs[1].hist(path_length, bins=20, color='mediumorchid', edgecolor='black')
    axs[1].axvline(observed_path_length, color='black', linestyle='--')
    axs[1].set_title(f"Average Path Length\nObserved is {observed_path_length:.4f}"
    axs[1].set_xlabel("Average path length")
    axs[1].set_ylabel("Frequency")

    plt.suptitle("Small-world models", fontsize=16, y=1.02)
    plt.show()
```

In [73]:
```
results_smallworld = generate_random_graph_stats(g, n_samples=10000, seed=42, exper
```

```
Loaded from data\smallworld_er_10000.pkl
```

In [74]:
```
observed_clustering = g.transitivity_undirected()
observed_path_length = g.average_path_length()

plot_smallworld_histograms(results_smallworld, observed_clustering, observed_path_l
```



As we see on the plots, the average path length is the same and the clustering coefficient is higher. This means that our network has small-world properties. Given the nature of the data, it was expected that the network would have small-world properties. The macaques are social animals, and they live in groups. In our analysis, we have seen how they have a lot of dominance interactions between them that lead to high clustering and how they are well-connected globally.

7. Conclusion

In this project, we have analyzed a network of dominance relations between macaques. We have seen how the network is highly connected, with a lot of interactions between the individuals. The network is dense locally and globally, with a lot of cliques and k-cores. In this framework, it is hard to make strong conclusions about the dominance relations between the individuals, as we do not have information about the direction of the edges. However, we can see that there are some individuals that are more central than others and that there are some communities in the network. This might induce us to think that some individuals with a lot of connections and interactions, like numbers 48 and 16, are dominant individuals and somehow are at the top of the hierarchy. We don't know how the macaque society works or if it is possible that two individuals are on top at the same time, so maybe one of them is dominant, and the other one is subordinate. In fact, when analyzing weighted edge betweenness, the second edge in importance is between those 2 macaques, indicating that this relationship is crucial in this macaque society. What we have seen when analyzing centrality is that in most metrics, node 48 is the most central, but when including the weights, 16 tends to be more important. This indicates that 16 is more active in those kinds of interactions. When seeing the degree distribution, we see how some have significantly higher than the others, while some have significantly lower. This indicates that while some have a lot of connections and interactions, others don't participate much in the dominance relations, most likely being the most subordinate individuals that avoid conflicts. The asorttativity coefficient is negative but could be considered neutral, which means that the dominance relations, even if they have a small tendency to be disassortative, are not really determined by the degree of the individuals.

When visualizing the network, we have seen how hard it is to see any pattern in such a dense network. However, we have seen that the MDS plot is the one that gives us the most information about the network. Also, by using some weights and colors, we can start to visualize some structures. Additionally, when using clustering algorithms, we see how the MDS representation of Fast Greedy, the most reliable algorithm according to our experiment, is the one that gives us the best representation of the network, as the clusters are well separated.

Finally, we have seen how the network has small-world properties, which is expected given the nature of the data. After comparing our network to random graphs, we can conclude that our network is more clustered than a random graph generated with the same number of vertices and edges and that the average path length is the same.